CARD PARTNER

Una web para amantes de los TCGS

Servicios y Sistemas Web

Laboratorio L2
GRUPO 33

ÁLVAREZ FULCONIS, MARCOS
LÓPEZ PRIMO, ÁLVARO
OTERO GARCÍA, SERGIO
SANTIESTEBAN MENDO, EDGAR

Curso 25-26

Índice

1. Presentación del equipo **1**
- 1.1. Organización del trabajo . . . 1
  - 1.1.1. Primeras dos entregas . . . 1
  - 1.1.2. Últimas dos entregas . . . 1
- 1.2. Justificación este cambio de enfoque . . . 2

2. Contexto y problema abordado **3**
- 2.1. Análisis de soluciones existentes . . . 4

3. *CardPartner* como idea general **5**

4. Usuarios objetivo **6**
- 4.1. Usuarios según su conocimiento del juego . . . 7
- 4.2. Usuarios según su nivel de acceso . . . 7

5. Especificación de requisitos **8**
- 5.1. Requisitos funcionales . . . 8
- 5.2. Requisitos no funcionales . . . 10
- 5.3. Requisitos de información . . . 11
- 5.4. Reglas de negocio . . . 11

6. Modelo del dominio **12**
- 6.1. Diagrama del Modelo del dominio . . . 12
- 6.2. Descripción de las entidades . . . 13
- 6.3. Usuarios y roles . . . 13
- 6.4. Cartas . . . 14
- 6.5. Artículo . . . 15
- 6.6. Mazos . . . 15
  - 6.6.1. Mazo de la comunidad . . . 16
  - 6.6.2. Mazo Oficial . . . 16
  - 6.6.3. Contiene Carta . . . 16
- 6.7. Interacción del usuario . . . 16
  - 6.7.1. Comentario . . . 16
  - 6.7.2. Valoración . . . 17
  - 6.7.3. Whislist . . . 17

7. Desarrollo de CardPartner **18**
- 7.1. Primera versión del bocetaje . . . 18
- 7.2. Segunda iteración sobre los bocetos . . . 19
  - 7.2.1. Vista inicial . . . 21
  - 7.2.2. Barra de navegación de la página . . . 22
  - 7.2.3. Comunidad . . . 23
  - 7.2.4. Mazos . . . 24
  - 7.2.5. Cartas . . . 25

8. Implementación de funcionalidades. **26**
- 8.1. Mapa del sitio web . . . 26
- 8.2. Estratificación por niveles del mapa de *Card Partner* . . . 27
  - 8.2.1. Nivel 1: *header* y navegabilidad . . . 27
- 8.3. Nivel 2: visualizadores y flujos verticales . . . 28
- 8.4. Flujos de navegación . . . 30
  - 8.4.1. Consulta de mazos META . . . 30

8.4.2. Consultar noticias . . . 30
8.4.3. Consulta de cartas específica . . . 30
8.4.4. Consultar los detalles de una carta incluida en un mazo . . . 30
8.4.5. Consulta de una de las guías de juego . . . 30
8.5. Base de datos . . . 30
8.6. Cambios del Modelo de Dominio a Entidad-Relación . . . 31
8.7. Estructura de la Base de Datos . . . 31
8.7.1. Componentes Principales . . . 31
8.8. Resumen de Relaciones y Cardinalidad . . . 32
8.9. APIs empleadas en nuestro sistema . . . 32
8.9.1. API encargada de la obtención de cartas . . . 32
8.9.2. API de traducción . . . 33
8.10. Descripción de la funcionalidad implementada y justificación de la elección . . . 33
8.10.1. Gestión de usuarios y perfil . . . 33
8.10.2. Catálogo de cartas y filtros . . . 33
8.10.3. Gestión y creación de mazos . . . 33
8.10.4. Comunidad e interacción . . . 33
8.10.5. Social . . . 34
8.10.6. Sección TCG . . . 34
8.11. Justificación de la elección . . . 34
8.12. Instrucciones de ejecución del código . . . 34
8.12.1. Usuarios para login . . . 35

9. Declaración de Uso de Inteligencia Artificial **36**

A. API de las cartas **37**

B. API de traducción **40**

1. Presentación del equipo

CardPartner es la idea de página web que hemos desarrollado el Grupo 33 para la asignatura de Sistemas y Servicios Web.
Este grupo está formado por Marcos Álvarez Fulconis, Álvaro López Primo, Sergio Otero García y Edgar Santiesteban Mendo.
La motivación de nuestra web es simple, ofrecer una herramienta intuitiva, de fácil uso y accesible para todas aquellas personas interesadas en adentrarse en el mundo de los *Trading Card Games* o TCGs, enfocándonos en el juego de cartas basado en el universo *de One Piece*.

1.1 Organización del trabajo

Debido a la amplitud del proyecto, hemos estado empleando un repositorio *Git* para poder paralelizar el trabajo. Hemos tenido dos fases de organización basándonos en la cantidad de tareas y el número de personas necesarias para cada una de ellas.

1.1.1 Primeras dos entregas

Durante las dos primeras entregas hemos optado por un enfoque mas grupal donde trabajábamos conjuntamente, ya que las primeras fases se basaban en maquetar el esqueleto de nuestra idea de página web.

Nombre	**Porcentaje**	**Responsabilidad principal**
ÁLVAREZ FULCONIS, MARCOS PALATINO	25 %	Diseño de las vistas tcg, social,noticias,torneo y guías
LOPEZ PRIMO, ÁLVARO	25 %	Diseño del index, login y todas sus opciones, creación del repositorio de github, redacción del documento
OTERO GARCÍA, SERGIO	25 %	Diseño de decks y todos sus submenús, *comunity decks, meta decks*, creación de mazos,vista de cartas
SANTIESTEBAN MENDO, EDGAR	25 %	Modelo de dominio,flujos de navegación, requisitos y redacción del documento

Cuadro 1: Distribución de responsabilidades y aporte de cada miembro en las dos primeras entregas de nuestra web *Card Partner*

De esta forma hemos conseguido plasmar los diferentes enfoques de cada uno de los integrantes del equipo, para así conseguir una web que cubra las diferentes necesidades que nos hemos ido encontrando a medida que investigábamos los *TCGs*

1.1.2 Últimas dos entregas

En las fases finales del proyecto decidimos cambiar el enfoque empleado: optamos por dividirnos en equipos de dos personas en cada uno de ellos para poder acelerar el trabajo y permitir más independencia a la hora de realizar toma de decisiones.
Los equipos se dividieron en base a como se ha realizado la implementación: *frontend* y *backend*.

Los compañeros del *frontend* se han encargado principalmente de la maquetación, el diseño visual, la navegación entre vistas y la implementación de las páginas JSP. Concretamente, han desarrollado vistas como la página principal, el catálogo de cartas, la visualización de las mismas, la sección de contenido social, el perfil de usuario, el login, el registro y todo lo relacionado con los mazos (la página de creación o construcción de los mismos, los mazos *META* o los mazos generados por la comunidad).

Por otra parte los compañeros del *backend* se ha centrado en el diseño e implementación de la bbdd, la creación de los scripts SQL, la definición de las clases del modelo, la construcción de los DAOs, la implementación de los Servlets y la automatización de la creación de la base de datos en local. También se ha encargado de transformar los datos necesarios para poblar la aplicación y de preparar la conexión entre la capa de presentación y la capa de persistencia.

Nombre	Porcentaje	Responsabilidad principal
Equipo Frontend		
Marcos Álvarez Fulconis	25 %	Desarrollo de vistas relacionadas con cartas y apartado social, apoyo en la navegabilidad general de la aplicación, revisión visual de las páginas y maquetación del documento.
Sergio Otero García	25 %	Desarrollo de vistas relacionadas con mazos, login, registro, página principal y constructor de mazos. Apoyo en la redacción del documento y en la integración visual de las distintas secciones.
Equipo Backend y Gestión de Datos		
Álvaro López Primo	25 %	Creación del esquema de la base de datos, colaboración en el modelo entidad-relación, población de la base de datos, transformación de datos externos y automatización de scripts para MariaDB en local. Apoyo en la redacción técnica del documento.
Edgar Santiesteban Mendo	25 %	Colaboración en el modelo entidad-relación, creación de esquemas de base de datos, desarrollo de clases del modelo, DAOs y DTOs, y apoyo en la redacción técnica del documento.

Cuadro 2: En esta tabla se refleja el enfoque de las últimas dos entregas y la aportación de cada uno de los integrantes del grupo 33

1.2 Justificación este cambio de enfoque

El cambio de nuestra manera de trabajar se debe a la amplitud del proyecto. Nuestra web requiere de una interfaz visual amplia, con múltiples vistas para cartas, mazos, perfil, noticias, guías y torneos, pero a su vez necesita una base de datos consistente y una lógica de servidor capaz de gestionar usuarios, cartas, mazos, comentarios, valoraciones y contenido editable.

Separar las responsabilidades nos ha permitido avanzar de forma más ordenada y nos ha ayudado a focalizar el trabajo individual, ya que cada uno de los equipos tenía una visión clara de la lista de tareas que debían realizar.

Esta metodología también nos ha facilitado la implemetación de mejoras y corrección de errores vistos en el *feedback* de entregas anteriores. Por ejemplo, hemos incorporado una vista de perfil del usuario más completa, hemos desarrollado mejor la distinción entre mazos de comunidad y mazos meta, y hemos consolidado la existencia de usuarios con distintos roles dentro del sistema. Además, la creación de scripts de automatización para la base de datos nos ha permitido que todos los miembros puedan trabajar con una estructura local equivalente, evitando problemas derivados de no disponer de un servidor remoto común.

2. Contexto y problema abordado

El mundo de los TCG, o *Trading Card Games*, ha crecido notablemente en los últimos años y cada vez reúne a más jugadores con perfiles muy distintos. Algunos usuarios se acercan por primera vez al juego y necesitan aprender sus reglas básicas, mientras que otros son eruditos de las mecánicas y buscan herramientas para consultar mazos competitivos, estudiar cartas, revisar precios o compartir sus propios mazos con la comunidad.

Gran parte del grupo 33 está comenzando a jugar a este tipo de juegos de cartas y nos hemos encontrado muchos problemas a la hora de introducirnos en este mundillo. El principal problema encontrado es la cantidad abrumadora de información que hay sobre estos juegos, añadiendo que pocos sitios web centralizan la información de una manera clara y accesible para personas novatas en estos *hobbies*. Ya no sólo basta con buscar información sobre el META (*Most Effective Tactics Available*), sino también el precio de las cartas que constituyen los mazos, donde se pueden adquirir las mismas o el circuito competitivo del mismo.

Por este motivo, hemos decidido plantear *Card Partner* como una aplicación web centralizada *All In One*, tanto para jugadores recién iniciados como veteranos aficionados a los TCG, reuniendo en un mismo sitio funcionalidades como la consulta de cartas, visualización de sus precios, creación y gestión de mazos, consulta de mazos META, noticias, guías, torneos y funcionalidades sociales como comentarios, valoraciones y mazos guardados.

Emplearemos como eje central el TCG basado en la serie japonesa *One Piece* por dos motivos: el primero es para acotar el problema, ya que existen cientos de juegos diferentes, cada uno con sus cartas y mecánicas, y si no tomamos uno como referencia para nuestra web tendríamos un problema inabarcable para 4 personas; el segundo es que es el TCG con el que más familiarizados estamos, gracias a que muchos de los miembros de este grupo somos, o jugadores experimentados en el juego o fanáticos del universo de la obra.

2.1 Análisis de soluciones existentes

Como ya hemos planteado en la sección anterior, ya existen aplicaciones web con un enfoque similar a lo que estamos planteando para esta entrega, por lo que hemos realizado un pequeño análisis de las mismas para ver sus fortalezas y debilidades.

Plataforma	**Fortalezas**	**Limitaciones detectadas**
Card Market	Es una de las plataformas más completas para consultar precios y realizar compraventa de cartas de distintos TCG.	Está orientada principalmente al mercado de cartas, por lo que no ofrece una experiencia centrada en la creación de mazos, el aprendizaje del juego o la interacción entre usuarios.
EggmanEvents	Permite consultar mazos utilizados en torneos y resulta útil para jugadores interesados en el entorno competitivo.	Su enfoque está muy ligado a resultados competitivos. Además, puede resultar menos intuitiva para usuarios novatos y no está orientada a que el usuario cree, gestione y comparta sus propios mazos.
One Piece Card Game	Es la fuente más fiable para consultar productos oficiales, reglas básicas y materiales introductorios del juego.	No funciona como comunidad de usuarios ni ofrece herramientas completas para construir mazos, valorar listas, guardar contenido personalizado o consultar precios de mercado.
One Piece Limitless	Reúne información competitiva, estadísticas de torneos, cartas y herramientas relacionadas con el juego.	Aunque ofrece información útil para jugadores avanzados, no centraliza en una misma experiencia el aprendizaje para principiantes, la gestión personal de mazos, los precios y la interacción social.

Cuadro 3: Comparación de plataformas existentes relacionadas con *One Piece* TCG

3. *CardPartner* como idea general

Esta revisión nos ha permitido tener muy claro el enfoque que le queremos dar a nuestra aplicación web con esa visión *All in One* que hemos comentado en el primer punto de este informe.

Nuestra aplicación permitirá a los usuarios realizar las siguientes acciones principales:

- **Consultar el catálogo de cartas:** navegar por todas las cartas existentes del juego.
- **Acceder al detalle de cada carta:** revisar información específica de una carta concreta, como el tipo, color, coste, poder y precio de mercado.
- **Crear mazos:** emplear el catálogo como base para la construcción de nuevas barajas jugables.
- **Explorar el módulo de mazos:** sección específica que clasifica las construcciones en dos categorías:
  - Los **mazos de la comunidad**, que representan listas creadas y compartidas por usuarios.
  - Los **mazos meta**, que permiten destacar construcciones competitivas o recomendadas por sus resultados.
- **Interactuar entre miembros de la comunidad:** posibilidad de que los usuarios puedan comentar, valorar y guardar los mazos de la plataforma.
- **Acceder a contenido editorial:** secciones dedicadas a noticias, guías y torneos, pensadas para mantener al usuario informado y facilitar la entrada de jugadores principiantes al TCG.
- **Gestionar la cuenta de usuario:** opciones para registrar cuentas, iniciar sesión, gestionar el perfil y acceder a funcionalidades personalizadas. Esto permite diferenciar la experiencia de un visitante no registrado de la de uno que sí lo está, puesto que estos últimos tienen la capacidad de crear, guardar y gestionar su propio contenido.

Con esto conseguimos que la aplicación sea útil tanto para jugadores con un perfil más competitivo o veteranos en este juego como para aquellos que no cuenten con tanta experiencia.

Llegados a este punto, nos podemos preguntar ¿qué aporta *Card Partner* al mercado de las aplicaciones web de TCGs?

La respuesta a esta cuestión es la siguiente: la interacción entre miembros de la comunidad en una web especializada, ya que es una *feature* muy poco desarrollada en este mundillo. Como equipo de desarrollo consideramos que el hecho de tener una aplicación útil es tan importante como el de tener una comunidad sana, que dispongan de una plataforma para ayudarse mutuamente, compartir conocimiento y valoraciones sobre mazos o líderes. El éxito de un TCGs no se basa en la calidad de los artes (las ilustraciones de las cartas) o la jugabilidad, si no en tener una comunidad activa la cual cuente con espacios para poder disfrutar y compartir su pasatiempo con todo aquel que comparta sus mismos gustos.

4. Usuarios objetivo

Teniendo en mente la idea y las funcionalidades que deberá tener nuestra aplicación web, vamos a analizar los usuarios objetivos para la misma. A continuación listaremos y añadiremos una pequeña descripción de los usuarios prototípicos de nuestra aplicación, aunque profundizaremos en cada uno de ellos en sus respectivos apartados más adelante. Consideramos que es importante tener una visión global de los mismos así como su clasificación.

Clasificación	**Perfil de Usuario**	**Descripción, Necesidades y Expectativas**
Según conocimiento del juego	**Principiante / Casual**	Jugador que recién comienza en el TCG. **Necesidades:** interfaz intuitiva, acceso fácil a guías, tutoriales y mazos recomendados para aprender a jugar. **Expectativas:** encontrar un entorno amigable que facilite, tanto por la información disponible como por la comunidad de la web,su curva de aprendizaje sin abrumarle con datos complejos.
	Experimentado / Competitivo	Jugador habitual que busca optimizar sus estrategias. **Necesidades:** filtros avanzados en el catálogo de cartas, acceso detallado a *mazos meta* y herramientas precisas para la construcción de sus propias barajas. **Expectativas:** información actualizada sobre el estado competitivo del juego.
Según nivel de acceso	**Visitante no registrado**	Usuario que accede a la plataforma de forma anónima. **Necesidades:** consultar el catálogo de cartas, leer noticias y ver mazos públicos de la comunidad o meta. **Expectativas:** navegación rápida y sin fricciones como primera toma de contacto con la aplicación.
	Jugador registrado	Usuario con cuenta activa en la plataforma. **Necesidades:** todas las del visitante no registrado, más la capacidad de crear, guardar y editar mazos, además de interactuar socialmente (comentar, valorar mazos de otros). **Expectativas:** disponer de un espacio personal seguro y persistente donde gestionar su progresión y creaciones, así como poder participar de la comunidad de los *TCGs*
	Administrador o Mantenimiento	*Perfil de backend.* Usuario encargado de la gestión integral de la plataforma. **Necesidades:** acceso a un panel de control (backoffice) para actualizar el catálogo de cartas (nuevas expansiones), redactar y publicar noticias/guías, y moderar el contenido de la comunidad. **Expectativas:** herramientas eficientes y seguras para garantizar que la plataforma esté siempre actualizada, estable y libre de contenido inapropiado.

Cuadro 4: Resumen y definición de los perfiles de usuario de *Card Partner*.

4.1 Usuarios según su conocimiento del juego

Esta primera definición nos permite diferenciar dos grandes grupos de usuarios, los *casuals* (termino empleado para definir a aquellas personas que conocen del juego pero no están interesados en el entorno competitivo) y los usarios competitivos o *tryhards*.

- Para los usuarios novatos hemos planteado secciones de guías, recomendaciones y mazos orientados a principiantes, de forma que puedan entender las reglas básicas, las mecánicas principales y las primeras decisiones necesarias para empezar a jugar.
- Para los jugadores experimentados **Card Partner** ofrece funcionalidades como el apartado de noticias, el catálogo de cartas, el constructor de mazos, los mazos de la comunidad, los mazos meta, los comentarios y las valoraciones. Con ello, hemos buscado facilitar tanto el seguimiento del entorno competitivo como la interacción entre jugadores.

4.2 Usuarios según su nivel de acceso

Además del tipo de usuario, tenemos que tener en cuenta las funcionalidades que va a poder realizar sobre nuestro sistema en función del nivel de acceso con el que cuente. Concretamente, tenemos 3 niveles diferentes, los visitantes (aquellos usuarios que no tienen cuenta), los que se encuentran registrados y por último los administradores de *Card Partner*.

- **Usuario NO registrado**: en esta clasificación encontramos a todos aquellos usuarios que no tienen una cuenta instanciada en nuestra aplicación web. Por este motivo, tienen acceso a funcionalidades limitadas, entre las que nos encontramos: consultar el catálogo de cartas, las noticias, guías, información sobre torneos, los mazos meta y los mazos publicados por la comunidad sin la posibilidad de interactuar con ellos (comentarlos, valorarlos, etc.). Su objetivo principal es explorar el contenido disponible antes de decidir si quiere registrarse.
  Pese a no tener acceso a todas las funcionalidades de nuestra web, su experiencia ha de ser satisfactoria puesto que el motivo para visitarnos es obtener información, ya sea de manera puntual o continuada.
- **Usuario SI registrado**: aquel que ha creado una cuenta e iniciado sesión en la aplicación. Este tipo de usuarios a mayores de las opciones disponibles para los no registrados cuenta con funcionalidades personalizadas, como la gestión del perfil, la creación de mazos, el guardado de mazos, la publicación de listas y la posibilidad de realizar comentarios así como la valoración de mazos creados por otros usuarios.
  Este tipo de usuario constituye el núcleo de la comunidad de Card Partner, puesto que toda la aplicación gira alrededor a estos (gracias al especial hincapié que hemos hecho en el apartado de mazos y al entorno social).
- **Administrador o usuario de mantenimiento**: representa al usuario con los permisos de gestión y mantenimiento dentro de la aplicación, gestionar el contenido de la web, publicar noticias, actualizar el catálogo de cartas o de mazos metas (tareas que, en ningún momento, puedan realizar los dos tipos de usuarios mencionados anterioremente).
  Entre sus funciones se encuentran la gestión de contenidos editoriales como noticias, guías y torneos; la revisión de información relevante para la plataforma, la actualización de mazos oficiales o meta y el mantenimiento general de los datos del sistema. De esta forma, el administrador actúa como responsable de que la información publicada sea coherente, esté actualizada y mantenga la calidad esperada.

5. Especificación de requisitos

En este apartado entraremos más en detalle sobre las características que vamos a desarrollar en nuestra aplicación web para poder satisfacer las demandas de los diferentes usuarios.

5.1 Requisitos funcionales

En esta sección describiremos las acciones que los usuarios pueden realizar dentro de la aplicación, manteniendo un organización por bloques temáticos para facilitar su lectura y poder relacionar cada requisito con su parte concreta dentro del sistema.

ID	Nombre	Descripción
1. Gestión de usuarios y perfiles		
RF-1.1	Registro	El sistema permitirá crear una cuenta de usuario mediante email y contraseña, o con una cuenta de Google
RF-1.2	Login	El sistema permitirá iniciar sesión para acceder a funcionalidades personalizadas.
RF-1.3	Credenciales	El sistema permitirá gestionar credenciales del usuario, incluyendo el cambio de contraseña desde el perfil o desde el acceso de recuperación.
RF-1.4	Perfil	El sistema permitirá consultar y gestionar un perfil de usuario con datos personales y mazos asociados.
2. Gestión y creación de mazos		
RF-2.1	Deckbuilder	El sistema permitirá crear mazos mediante una herramienta interactiva de selección de carta líder y cartas del mazo.
RF-2.2	Almacenamiento	El sistema permitirá guardar mazos asociados a un usuario registrado.
RF-2.3	Publicación	El sistema permitirá publicar mazos para que puedan ser consultados por otros usuarios de la comunidad.
RF-2.4	Biblioteca	El sistema permitirá mostrar los mazos creados, publicados y guardados por el usuario.
RF-2.5	Filtro Comunidad	El sistema permitirá buscar o filtrar mazos de la comunidad según criterios como líder, nombre o características relevantes.
RF-2.6	Estadísticas	El sistema permitirá visualizar datos de rendimiento asociados a mazos meta, como el winrate.
3. Base de datos de cartas		
RF-3.1	Catálogo	El sistema permitirá consultar una vista general de las cartas disponibles.
RF-3.2	Filtro avanzado	El sistema permitirá buscar cartas por atributos como nombre, color, tipo, coste o rareza, según la información disponible.
RF-3.3	Precios	El sistema permitirá consultar el precio de mercado asociado a las cartas.
RF-3.4	Detalle de carta	El sistema permitirá acceder a una vista detallada de una carta con sus atributos principales.
4. Comunidad e interacción		
RF-4.1	Comentarios	El sistema permitirá a los usuarios registrados comentar mazos publicados por otros usuarios.
RF-4.2	Valoración	El sistema permitirá valorar mazos para destacar aquellos mejor recibidos por la comunidad.
RF-4.3	Mazos guardados	El sistema permitirá guardar mazos para consultarlos posteriormente desde el perfil del usuario.
RF-4.4	Wishlist	El sistema permitirá asociar cartas a una lista de deseos del usuario.

5. Contenido editorial y recursos		
RF-5.1	Noticias	El sistema permitirá consultar noticias y novedades relacionadas con el TCG.
RF-5.2	Guías	El sistema permitirá consultar guías básicas para facilitar el aprendizaje de jugadores principiantes.
RF-5.3	Torneos	El sistema permitirá consultar información o enlaces relacionados con eventos competitivos.
RF-5.4	Mazos recomendados	El sistema permitirá mostrar mazos recomendados para principiantes dentro de la página principal o de secciones de apoyo al usuario novel.
6. Administración y mantenimiento		
RF-6.1	Gestión de contenido	El sistema permitirá que usuarios administradores gestionen noticias, guías, torneos y contenido editorial.
RF-6.2	Gestión de mazos oficiales	El sistema permitirá distinguir mazos oficiales o meta frente a mazos creados por la comunidad.
RF-6.3	Mantenimiento de datos	El sistema permitirá mantener actualizada la información necesaria para el correcto funcionamiento de la plataforma.

5.2 Requisitos no funcionales

Sección dedicada a las condiciones de calidad que debe cumplir la aplicación, adaptando conceptos como rendimiento, usabilidad, seguridad, mantenibilidad y compatibilidad, adaptándolos al alcance real del proyecto.

ID	Nombre	Descripción
1. Rendimiento y eficiencia		
RNF-1.1	Tiempo de respuesta	El sistema deberá ofrecer tiempos de carga razonables en las vistas principales y en las búsquedas de cartas y mazos.
RNF-1.2	Optimización	El sistema deberá evitar cargas innecesarias de recursos, especialmente en imágenes de cartas y listados extensos.
RNF-1.3	Paginación	El sistema deberá limitar la cantidad de cartas o mazos mostrados simultáneamente para mejorar la navegación.
2. Usabilidad y experiencia de usuario		
RNF-2.1	Diseño coherente	La interfaz deberá mantener una apariencia homogénea entre las distintas vistas.
RNF-2.2	Navegación	El usuario deberá poder acceder a las secciones principales desde una cabecera común.
RNF-2.3	Aprendizaje	El constructor de mazos deberá ser comprensible para usuarios que no tengan experiencia previa con herramientas similares.
3. Seguridad y privacidad		
RNF-3.1	Protección de datos	El sistema deberá proteger la información personal de los usuarios y evitar exponer datos sensibles innecesariamente.
RNF-3.2	Control de acceso	Las funcionalidades personalizadas deberán requerir que el usuario haya iniciado sesión.
RNF-3.3	Validación	El sistema deberá validar los datos recibidos desde formularios para reducir errores e inserciones no válidas.
4. Disponibilidad y mantenibilidad		
RNF-4.1	Base de datos local reproducible	El sistema deberá poder desplegar su base de datos en local mediante scripts de creación e inserción.
RNF-4.2	Modularidad	La aplicación deberá mantener una separación clara entre vistas JSP, Servlets, DAOs, modelos y base de datos.
RNF-4.3	Actualización	La estructura deberá facilitar la incorporación de nuevas cartas, mazos o artículos sin rediseñar por completo el sistema.
5. Compatibilidad y portabilidad		
RNF-5.1	Navegadores	La aplicación deberá ser accesible desde navegadores actuales como Chrome, Firefox, Edge o Safari.
RNF-5.2	Entorno de ejecución	La aplicación deberá poder ejecutarse en un entorno local con Apache NetBeans, Tomcat y MariaDB.
RNF-5.3	Automatización	El proyecto deberá incluir scripts que faciliten la creación de la base de datos en distintos sistemas operativos.

5.3 Requisitos de información

En este puntos nos centramos en describir los datos que el sistema debe almacenar y gestionar para que las funcionalidades anteriores puedan llevarse a cabo. Hemos definido estos requisitos a partir del modelo de dominio (punto 6) y de la estructura de la base de datos implementada(punto **??**).

ID	Entidad	Información gestionada
RI-1	Usuario	El sistema almacenará identificador, apodo, email, contraseña, avatar, rol y fecha de registro de cada usuario.
RI-2	Carta	El sistema almacenará identificador oficial, nombre, imagen, tipo, arquetipo, color, coste, poder, atributo y precio de cada carta.
RI-3	Mazo	El sistema almacenará identificador, nombre, descripción, fecha de creación, visibilidad, oficialidad, winrate, usuario creador y carta líder.
RI-4	Contenido de mazo	El sistema almacenará la relación entre mazos y cartas, incluyendo la cantidad de copias de cada carta dentro del mazo.
RI-5	Comentario	El sistema almacenará contenido, fecha de creación, fecha de edición, usuario autor y mazo al que pertenece el comentario.
RI-6	Valoración	El sistema almacenará la puntuación que un usuario asigna a un mazo y la fecha de dicha valoración.
RI-7	Artículo	El sistema almacenará título, descripción, categoría, fecha de creación, fecha de edición, enlace externo, portada y administrador autor.
RI-8	Mazo guardado	El sistema almacenará la relación entre usuarios y mazos guardados.
RI-9	Wishlist	El sistema almacenará la relación entre usuarios y cartas añadidas a su lista de deseos.

5.4 Reglas de negocio

Llegados a este punto sólo nos queda mencionar las restricciones propias del dominio que deben cumplirse para que la información gestionada sea coherente. Hemos incluido estas reglas para concretar mejor el comportamiento esperado del sistema y evitar ambigüedades en la gestión de usuarios, cartas y mazos.

ID	Regla de negocio
RN-1	Todo usuario registrado debe tener un email único, un apodo identificable y un rol asociado dentro del sistema.
RN-2	Solo los usuarios registrados pueden crear mazos, guardar mazos, comentar, valorar o añadir cartas a la wishlist.
RN-3	Todo mazo debe tener una carta líder asociada.
RN-4	Todo mazo debe estar asociado a un usuario creador.
RN-5	Un mazo puede ser público o privado, según decida su creador.
RN-6	Un mazo puede ser de comunidad u oficial/meta. Esta distinción se representa mediante el atributo de oficialidad del mazo.
RN-7	La relación entre mazo y carta debe almacenar la cantidad de copias de cada carta incluida.
RN-8	Las cartas deben identificarse mediante su código oficial para facilitar su relación con fuentes externas y evitar duplicados.
RN-9	Los comentarios deben almacenar tanto la fecha de creación como la fecha de edición para diferenciar cuándo se publicaron y cuándo fueron modificados.
RN-10	Cada artículo debe pertenecer a una categoría concreta: noticia, guía o torneo.
RN-11	Los mazos guardados por un usuario podrán corresponder a mazos públicos de la comunidad o a mazos oficiales/meta visibles en la aplicación.
RN-12	Las funcionalidades de administración y mantenimiento quedan reservadas a usuarios con rol administrativo.

6. Modelo del dominio

A lo largo de esta sección haremos representación visual y conceptual de las entidades principales que conforman *CardPartner*. Buscamos ofrecer una visión clara de la estructura de la información y la lógica subyacente, abstrayéndonos de los detalles técnicos de implementación puesto que se tratarán en puntos posteriores de este informe.

6.1 Diagrama del Modelo del dominio

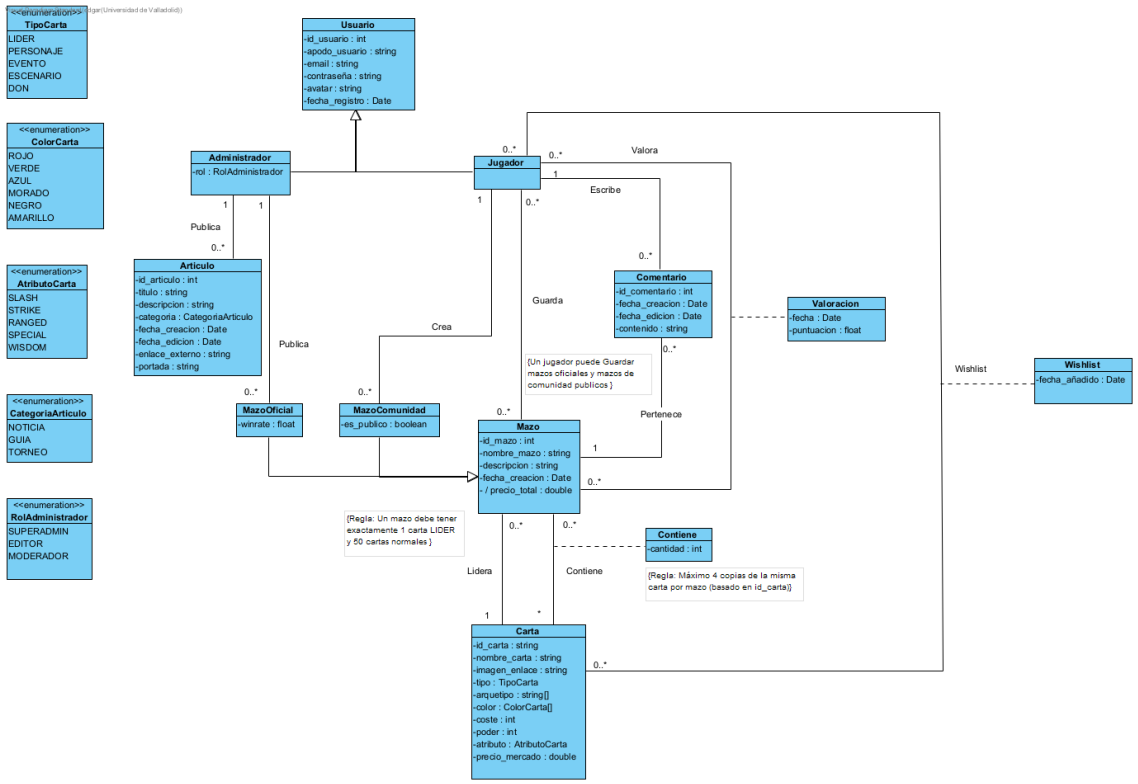


Figura 1: Diagrama UML del Modelo de Dominio de Card Partner donde definimo los conceptos clave de las reglas de negocio.

6.2 Descripción de las entidades

Teniendo la visión general de la aplicación, vamos a ir profundizando en cada una de las entidades. Emplearemos pequeños diagramas, así como una explicación de sus atributos y las relaciones guardan entre ellas.

6.3 Usuarios y roles

La clase Usuario representa la información de acceso y autenticación del usuario registrado en la web.

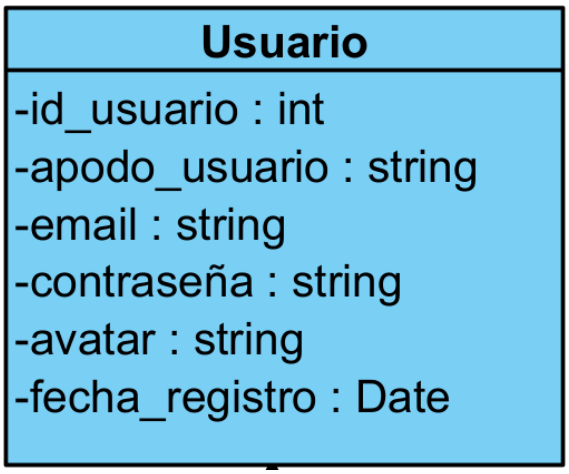


Figura 2: Entidad Usuario

Atributos:

- `id_usuario`: identificador único del sistema.
- `apodo_usuario`: nombre público elegido por el usuario.
- `email`: cuenta de correo electrónico asociada a la cuenta.
- `contraseña`: credencial del usuario (cifrada).
- `fecha_registro`: fecha de creación de la cuenta.
- `avatar`: ruta a la imagen pública del perfil del usuario.
- `rol`: tipo de rol que determina los privilegios dentro de la web..

La relaciones que puede tener esta entidad son las siguientes:

- **Crea(n:m)** con MazoComunidad : el Usuario puede crear mazos y publicarlos en la web.
- **Guarda(n:m)** con Mazo : el Usuario puede guardar únicamente mazos visibles, es decir, mazos públicos de la comunidad o mazos oficiales/meta. No puede guardar mazos privados de otros usuarios.
- **Escribe (1:n)** con Comentario : el Usuario puede publicar comentarios en los mazos.
- **Valora (n:m)** con Mazo : el Usuario puede dar una valoración en los mazos.
- **Publica(n:m)** con MazoOficial : sólo usuarios con rol ADMIN pueden publicar mazos oficiales/meta.
- **Publica(n:m)** con Articulo : solo usuarios con rol ADMIN pueden publicar artículos (Noticias, Guías o Torneos).

6.4 Cartas

La clase Carta representa la entidad central del TCG y su información estática.

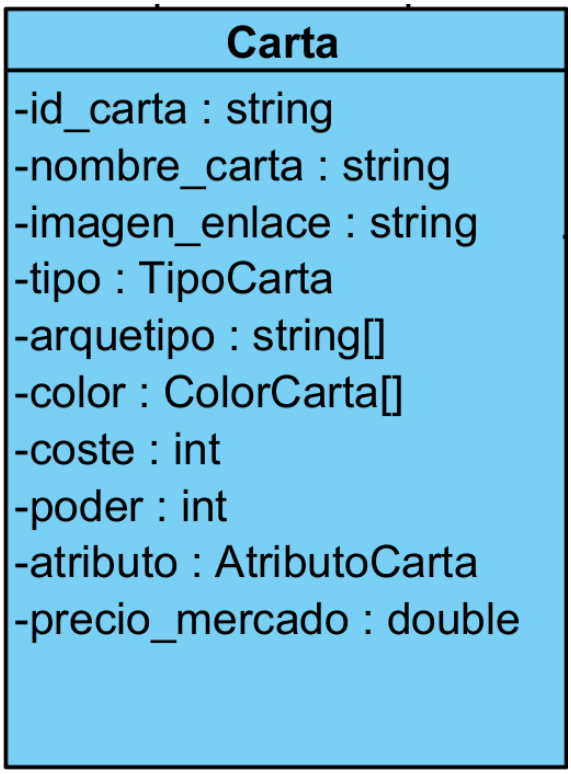


Figura 3: Entidad Carta

Atributos:

- id_carta : código oficial de la carta.
- nombre_carta : nombre de la carta.
- imagen_enlace : ruta a la ilustración de la carta
- tipo : clasificación de la carta dentro del TCG de *One Piece*.
- arquetipo : familia a la que pertenece la carta (pueden ser varias).
- color : clasificación de la carta dentro del TCG de *One Piece*.
- coste : puntos necesarios para jugar la carta.
- poder : puntos de ataque de la carta.
- atributo :tipo de ataque de la carta.
- precio_mercado : valor real de la carta.

Las relaciones que presenta esta entidad son:

- **Lidera(1:1)** con Mazo : identifica a la única carta líder obligatoria de un mazo.
- **ContieneCarta(n:m)** con Mazo : identifica las cartas que componen un mazo, con cantidad.

6.5 Artículo

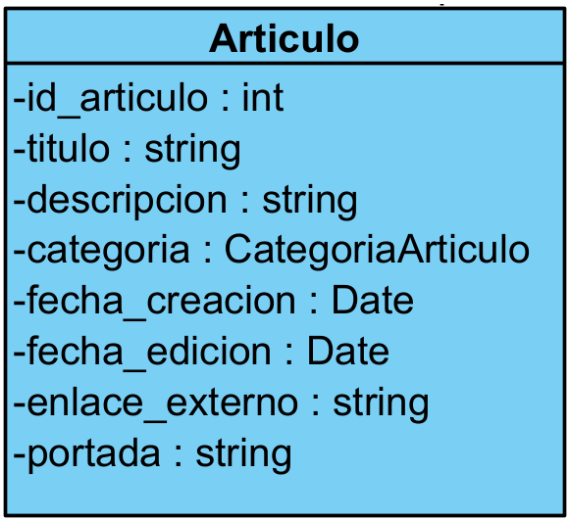


Figura 4: Contenidos publicados por Usuarios con rol ADMIN.

Atributos:

- id_articulo : identificador único.
- titulo : titular del artículo.
- descripcion : resumen del contenido.
- categoria : tipo de artículo (Noticia, Guia, Torneo).
- fecha_creacion : fecha de creación.
- fecha_edicion : fecha de última edición.
- enlace_externo : URL de referencia.
- portada : ruta de imagen miniatura

6.6 Mazos

La entidad Mazo representa la estructura común de cualquier mazo publicado en la web.

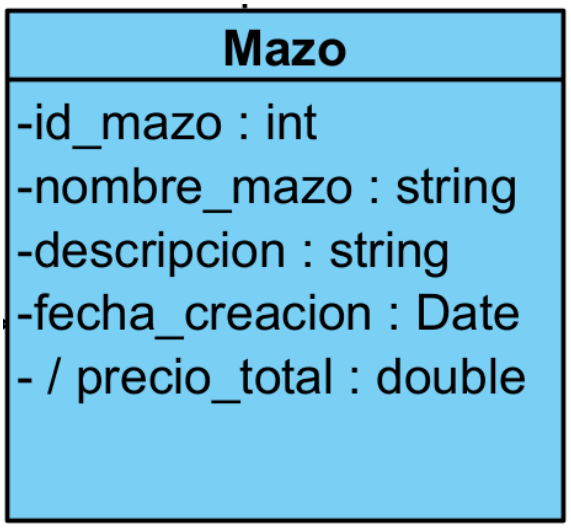


Figura 5: Entidad Mazo

Atributos:

- id_mazo : identificador único del sistema.
- nombre_mazo : título asignado por el creador.
- descripcion : texto explicativo del creador.
- fecha_creacion : fecha de creación del mazo.
- precio_total : precio total de todas las cartas que lo componen.

Mazo presenta las siguientes relaciones:

- Usuario: guarda y valora.
- Carta: lidera y contieneCarta.
- Comentario: pertenece.

6.6.1 Mazo de la comunidad

La clase MazoComunidad representa aquellos mazos que han sido creados por un Usuario (con rol JUGADOR). Dicha entidad presenta los siguientes atributos con sus respectivas relaciones

Atributos:

- `es_publico`: identifica si el usuario creador ha hecho el mazo público o es privado.

Relaciones:

- **Usuario:** crea.

6.6.2 Mazo Oficial

Esta clase representa aquellos mazos creados por un Usuario con rol ADMIN y contiene datos verificados del entorno competitivo.

Atributos:

- *winrate* : porcentaje de victorias del mazo en competiciones oficiales

Relaciones:

- Usuario: publica

6.6.3 Contiene Carta

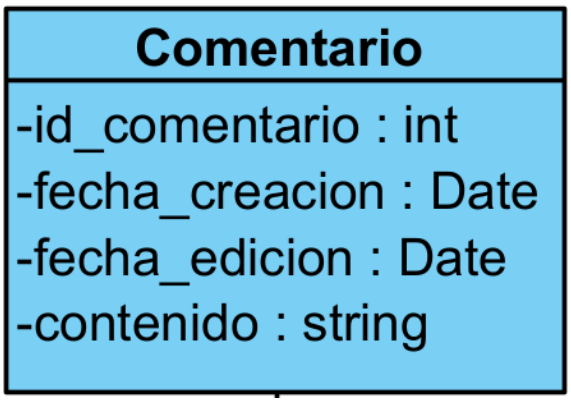
Esta clase asociativa único atributo es **cantidad** que representa el número de copias de la carta dentro del mazo, ya que es la única información que aporta.

6.7 Interacción del usuario

En este apartado explicaremos más en profundidad las entidades que representan interacciones directas del usuario en el aspecto social.

6.7.1 Comentario

Entidad que representa la publicación que un Usuario deja en un Mazo.



Atributos:

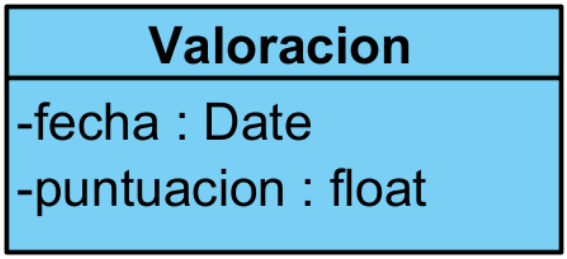
- id_comentario : identificador único.
- contenido : texto redactado por el usuario.
- fecha_creacion : fecha de creación del comentario.
- fecha_edicion : fecha de última edición.

Figura 6: Entidad comentario.

Esta entidad sólo tiene una relación con la entidad mazo, la cual es de pertenencia.

6.7.2 Valoración

Clase asociativa entre Usuario y Mazo.



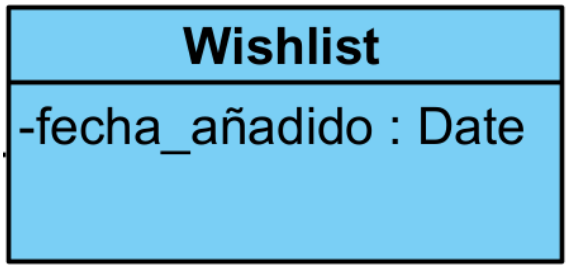
Atributos:

- fecha_valoracion : fecha de la valoración.
- puntuacion : valor numérico asignado.

Figura 7: Entidad valoración.

6.7.3 Whislist

Esta entidad actúa como una clase asociativa que representa la interacción de guardado entre un Jugador y una carta.



Atributos:

- fecha_añadido:

Figura 8: Entidad *whishlist*

Whishlist presenta las siguientes relaciones:

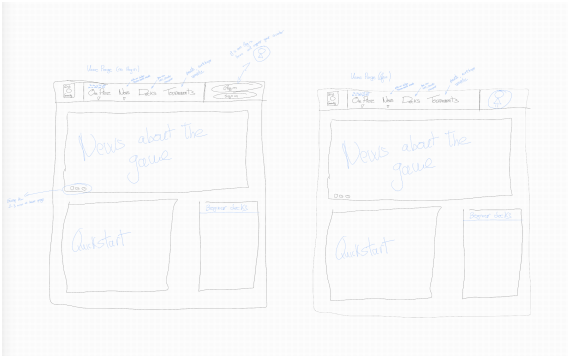
- Usuario: el cual ha de tener una sesión inciada en nuestra aplicación
- Carta: la entidad que el usuario quiere añadir a su lista de deseados.

7. Desarrollo de CardPartner

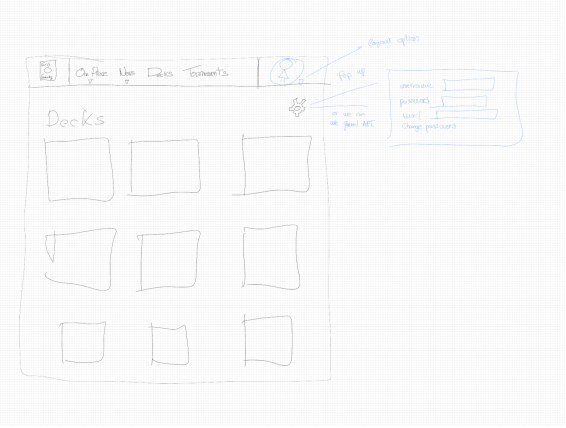
Llegados a este punto, ya tenemos establecidos unos planos sólidos sobre cómo ha de ser nuestra aplicación web, por lo que a partir de este punto documentaremos las diferentes etapas en el desarrollo del proyecto, partiendo desde el bocetaje incial, pasando por el desarrollo del *frontent* estático, la posterior incorporación del *backend* y por último la funcionalidad definitiva para hacer pasar a *Card Partner* de una idea a una web funcional.

7.1 Primera versión del bocetaje

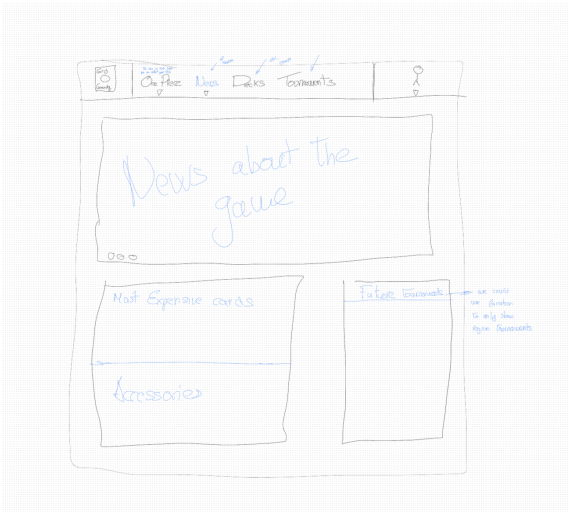
En la primera fase hemos realizado unos esquemas a mano alzada sobre el aspecto y funcionalidades de la web (para consultar los bocetos con más detalle puesto que no son imágenes vectorizadas diríjase a este enlace).



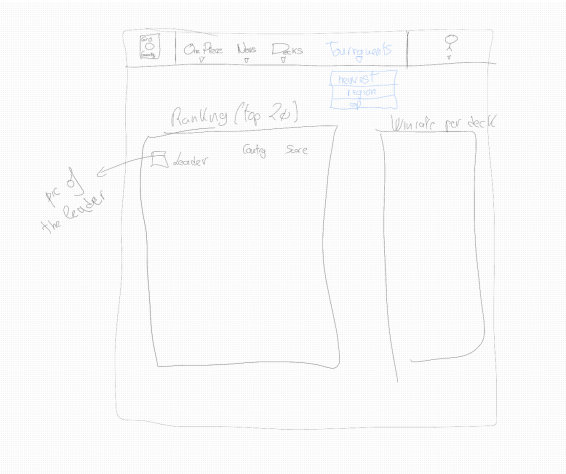
(a) Vista inicial de la página con la opción de *login* y con una cuenta ya creada donde se verán unas breves noticias sobre el *TCG*, un apartado de las normas básicas del juego y una sección de mazos recomendados para principiantes.



(b) Vista del usuario donde podrá ver los mazos que ha creado y una rueda de opción donde poder realizar diferentes funciones sobre su perfil (cambiar contraseña)



(a) Vista del apartado de noticias. En ella observamos novedades del *TCG* así como una selección de cartas más caras y de accesorios típicos para dichos juegos así como una lista de los futuros torneos.



(b) Vista de la pestaña de torneos. Incluye un ranking global de mazos y un porcentaje de victoria de cada líder.

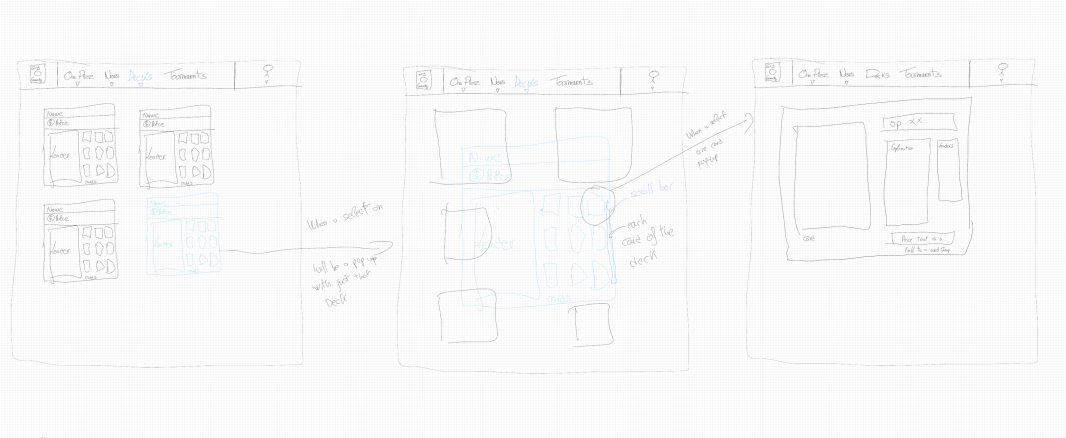


Figura 11: Vista del apartado Deck. Se muestran los diferentes mazos creados por la comunidad, con la opción de seleccionar sobre ellos y poder consultar todas las cartas. Además se incluye una opción a las personas con una cuenta en la web de comentar en los mazos

7.2 Segunda iteración sobre los bocetos

Una vez que todos hemos dado el visto bueno a los bocetos a mano alzada, decidimos emplear la herramienta *draw.io* para realizar unos bocetos más limpios y organizados (para consultar las imágenes con más detalle, diríjase a este enlace)
Esta mejora en la organización se debe a que hemos dividido los bocetos en módulos, los cuales se pueden consultar en la siguiente tabla

Módulo principal	Funcionalidades y características
Comunidad	Acceso a guías de aprendizaje, noticias sobre el juego y seguimiento de torneos.
Mazos	Consulta del meta actual con estadísticas, acceso a mazos de la comunidad (con filtros, valoraciones y comentarios) y una herramienta interactiva para la creación de mazos propios.
Cartas	Catálogo completo con listado de cartas, filtros de búsqueda avanzados, estadísticas detalladas y precios de mercado.
Usuario	Gestión integral de la cuenta: inicio de sesión, registro, edición de información personal (correo, contraseña) y un panel de control para administrar mazos (creados, publicados y guardados) e interacciones sociales.

Cuadro 5: Descripción general de los módulos y funcionalidades de la plataforma.

En la siguiente página podemos observar los nuevos bocetos, más estructurados y profesionales realizados con la herramienta *draw.io*.

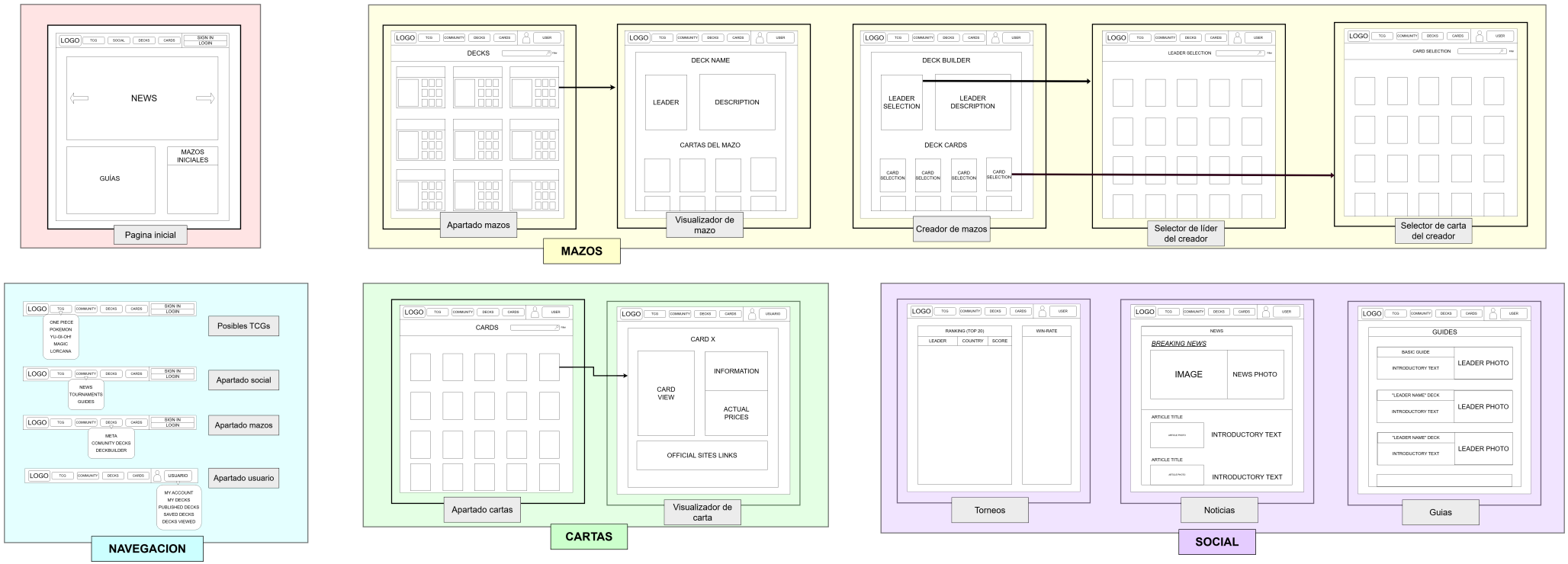


Figura 12: Vista completa de los bocetos de Draw.io

7.2.1 Vista inicial

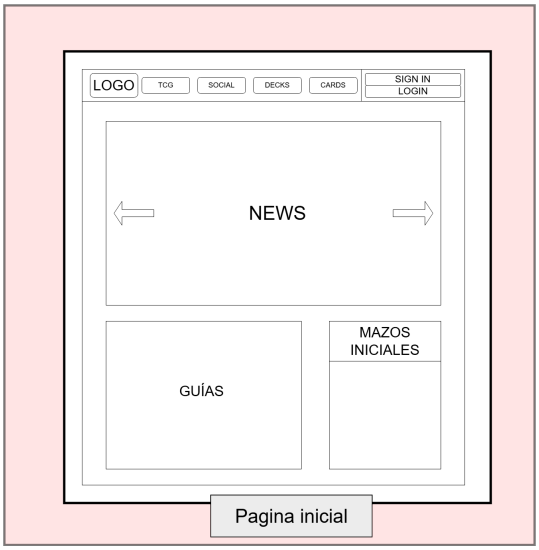


Figura 13: **Boceto de la página de Inicio**

Observamos que la página inicial se compone de dos elementos clave, la barra de navegación (explicada en la siguiente captura) y los contenidos más relevantes. Enfocándonos en los contenidos relevantes, hemos planteado que se incluya un subapartado de novedades (nuevas expansiones disponibles, información destacada sobre torneos, noticias relacionadas con el *TCG*, etc.), un subapartado con las guías para principiantes y un subapartado con los mejores mazos con los que comenzar a jugar.

7.2.2 Barra de navegación de la página

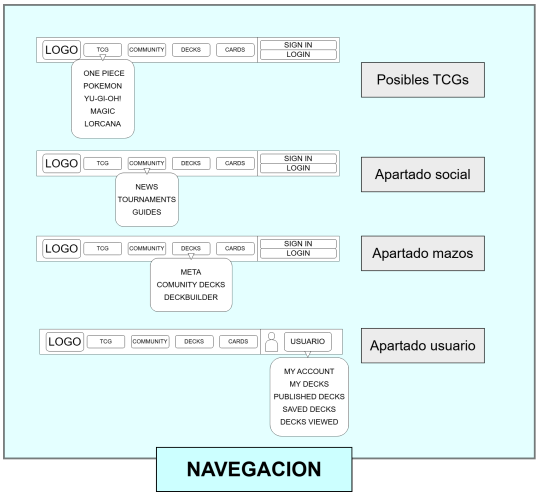


Figura 14: **Boceto de la barra de navegación**

La barra de navegación es el motor de navegabilidad principal de nuestra aplicación web, ya que permite al usuario desplazarse por las diferentes funcionalidades de la web de una manera simple y organizada, donde la información está categorizada por menús generales que contienen las diferentes vistas con las que está relacionada, permitiendo al usuario llegar a la opción deseada con un leve movimiento del ratón.
Para hablar sobre la navegación podemos dividir la barra en dos secciones: sección del juego y sección del usuario.

En la sección de **juego** observamos, de izquierda a derecha, las siguientes categorías: *TCG*, *Community* (comunidad), *Decks* (mazos) y *Cards* (cartas).
La categoría **TCG** hace referencia a los principales juegos que existen (como hemos mencionado anteriormente nos enfocaremos en el juego basado en *One Piece*). En comunidad podemos acceder a las vistas relacionadas con la interacción del usuario en la comunidad: noticias, torneos y guías de usuario. La siguiente categoría, **Decks**, incluye todo aquello relacionado con mazos: Mazos empleados en el *META*, mazos creados por la comunidad y el creador de mazos del usuario. Por último nos encontramos con la categoría *Cards*, la cual nos lleva a la lista completa de cartas existentes.

Por otro lado nos encontramos la sección del **usuario**. Inicialmente vemos dos opciones, la opción de *Crear cuenta*, pensada para miembros sin un perfil registrado en nuestra web; y la opción de *Iniciar sesión*, enfocada a que los miembros puedan entrar a sus respectivas cuentas. Una vez iniciada la sesión cambiarán los botones a uno que despliegue un menú con las diferentes opciones: Mi cuenta, mazos creados, mazos publicados, mazos guardados e interacciones.

7.2.3 Comunidad

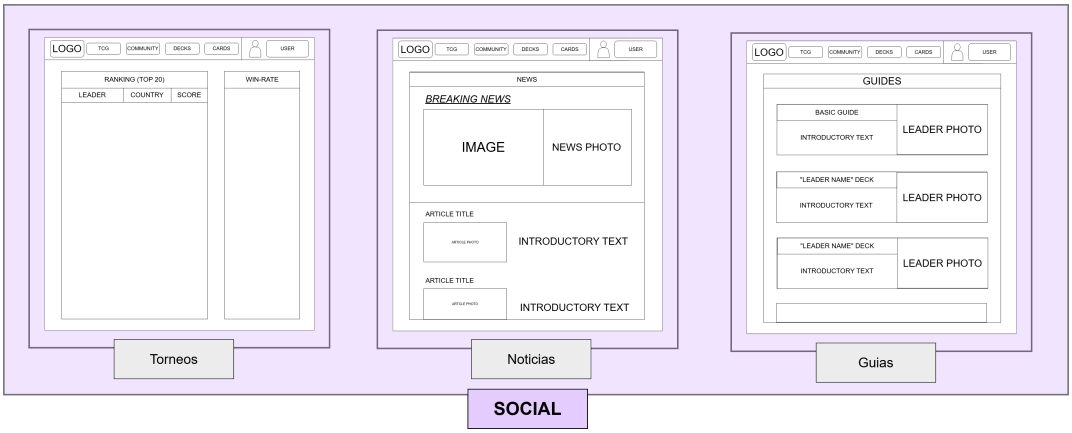


Figura 15: **Boceto de las pestañas de la comunidad**

Nos adentramos ahora en la sección de comunidad. Esta estará dividida en tres opciones diferentes: torneos, noticias y guías.

En el apartado de torneos tendremos una lista con los diferentes eventos que hay (torneos pasados o futuros).
Al seleccionar un torneo entramos en el visualizador del torneo, el cual se compone de dos listas, la lista de *Ranking*, que muestra las posiciones de los diferentes mazos participantes en orden de su desempeño en la competición; y el apartado de *Win-rate*, donde nos informará con una lista sobre el porcentaje de victorias que ha tenido cada carta líder participante.

Pasamos ahora a las noticias. En esta vista nos encontramos con una lista que incluye las noticias relacionadas con el mundo del *TCG*. En la parte superior veremos las noticias con mayor relevancia, y en la parte inferior el listado con todas las noticias ordenadas por fecha más reciente.
Cada noticia incluye un título, una imagen y una pequeña sinopsis. Al hacer click en ella veremos la noticia elegida en su totalidad.

Muy similar al apartado anterior, el de guias se compone de una lista con todas las guias publicadas hasta el momento. Cada noticia tiene un nombre, una breve explicación y una imagen relacionada con ella. Al seleccionar una guia se nos abrirá un visualizador completo de la guia, con sus instrucciones, imagenes explicativas e incluso vídeos informativos.

7.2.4 Mazos

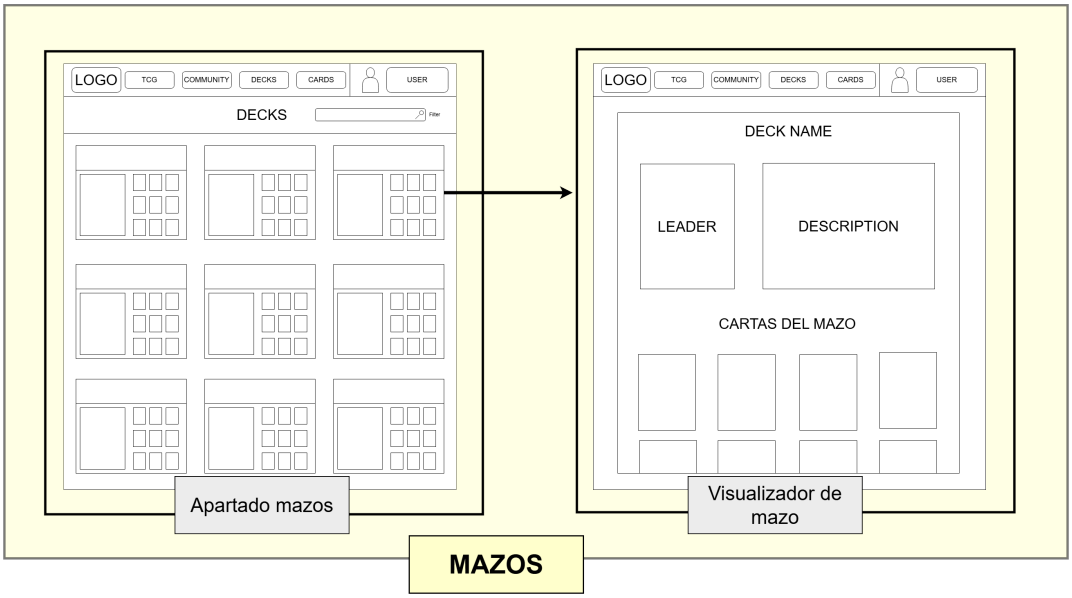


Figura 16: **Boceto de la pestaña de mazos**

Esta pestaña incluye una lista con los elementos visuales de los diferentes mazos, tanto mazos *META* introducidos por los administradores como los creados por la comunidad y los creados por el propio usuario, añadiendo también la opción de filtrar la búsqueda por elementos clave o buscar por nombre. Seleccionando un mazo concreto se nos abrirá el visualizador del mazo. Aquí podemos obtener información en detalle del mismo: el nombre del mazo, la carta *Líder* del mazo, una descripción de este y la lista de cartas que lo compone.

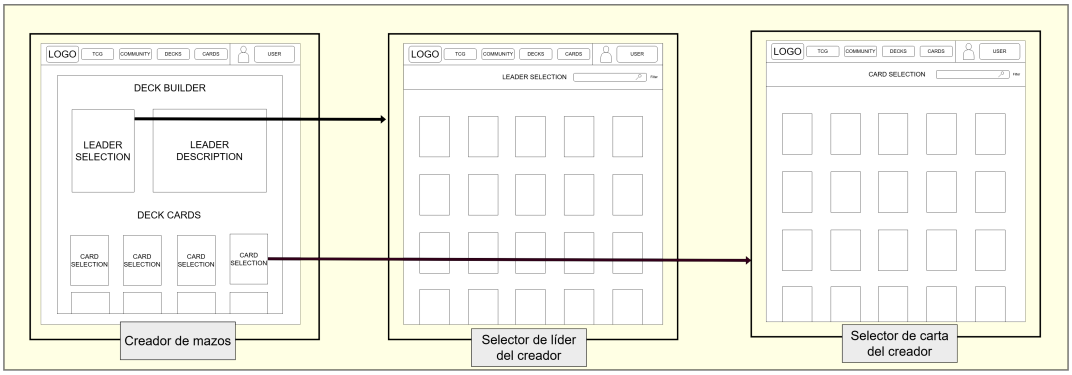


Figura 17: **Boceto de la pestaña del creador mazos**

Ahora observamos el apartado del creador de mazos. La vista es similar a la visualización de mazos de la comunidad, con la diferencia de que inicialmente no habrá cartas ni información de ningún tipo.

Al seleccionar el apartado de la carta *Líder* se nos abrirá un selector de cartas muy parecido a la lista entera de cartas (explicado en el siguiente apartado). Se pedirá al usuario que seleccione el color en el que esté basado el mazo que quiere crear (dentro del mundo del juego de One Piece hay cartas con diferentes colores, cada color tiene una mécanica diferente) y a raíz de eso saldrá la lista de líderes filtrada por el color seleccionado.

De la misma forma se abrirá una lista de cartas normales al seleccionar el apartado de cartas normales, filtradas ya con el color del líder.

7.2.5 Cartas

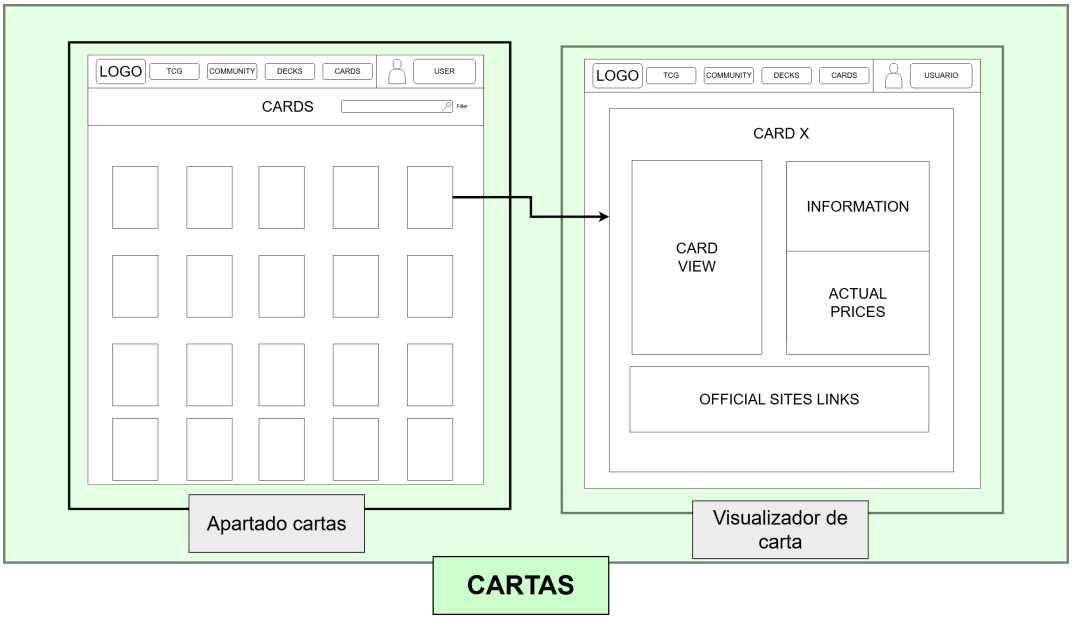


Figura 18: **Boceto de la pestaña de cartas**

Similar a la pestaña de mazos, la vista de cartas nos incluye la lista de todas las cartas existentes. Encontramos diversos filtros de búsqueda (color, coste, daño, mecánica, etc.) que nos ayudan a encontrar las cartas que más convengan al usuario. Si se selecciona una carta podemos visualizar información en detalle: El nombre de la carta, una imagen de ella, el texto de información que está incluido en la carta, el precio en la web de compraventa y links a la página oficial del juego relacionados con ella.

8. Implementación de funcionalidades.

En este punto del desarrollo pasamos de una web estática a una aplicación web basada en JSP, Servlets, DAOs, clases de dominio y una base de datos relacional en MariaDB. A partir de este punto vamos a ir añadiendo y documentando la parte dinámica de nuestra web, es decir, la funcionalidad.

8.1 Mapa del sitio web

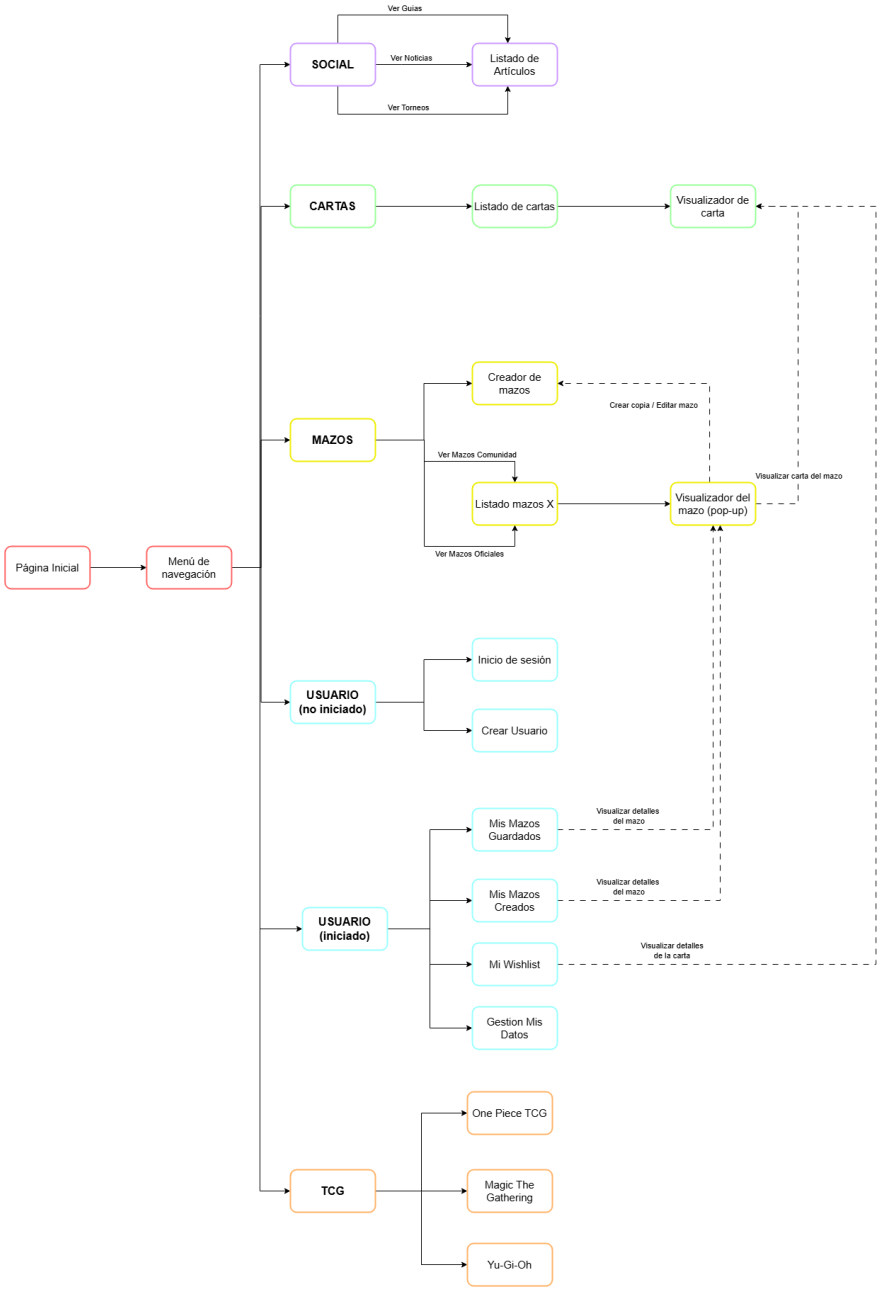


Figura 19: Estructura jerárquica y mapa del sitio web de Card Partner.

Nuestro diseño está centrado en la navegabilidad, permitiendo un acceso rápido a las diferentes funcionalidades del sitio web desde el *header* de la web.

8.2 Estratificación por niveles del mapa de *Card Partner*

Para facilitar la comprensión del mapa mostrado, vamos a añadir una explicación concisa sobre la arquitectura de la información que va a usar nuestra aplicación, dividida en niveles en función de los clicks o interacciones necesarias que tiene que realizar el usuario para llegar a dicha funcionalidad.
Partiremos desde el **nivel 0** o **el punto de entrada**. Es aquél en el que se encuentra la **página principal**, el punto de acceso de todo usuario a nuestra aplicación y a su vez el menú que permitirá el acceso al resto de apartados.

8.2.1 Nivel 1: *header* y navegabilidad

El *header*: es el motor de navegabilidad principal de nuestra web. Se encuentra en la parte superior de cada una de las vistas y nos permite dividir la web en las 4 grandes secciones que constituyen el siguiente nivel, de esta manera el usuario final podrá navegar por las diferentes funcionalidades de la web de manera cómoda e intuitiva.

Esto nos da acceso a todas las **herramientas y listados**:

- **SOCIAL:** apartado donde el usuario final podrá encontrar noticias, artículos y guías sobre el juego. Se mostrarán 3 subsecciones como menús desplegables, los cuales permitirán acceder a cada una de estas vistas. Estas vistas ueden describirse como el **Listado de Artículos**, que mostrará contenidos de categorías de artículos diferentes (Noticias, Guías o Torneos) según a cual se acceda. Haciendo click en el artículo se redirigirá al usuario al sitio externo referido del artículo.
- **CARTAS:** esta opción da acceso al **listado de cartas** del TCG, donde el usuario podrá clickar sobre cada una de ellas para así poder acceder a una vista detallada de la seleccionada, dirigiendo al usuario al **Visualizador de carta**.
- **MAZOS:** apartado a que agrupa todas las herramientas relacionadas a los mazos, permitiendo al usuario realizar acciones sobre este conjunto de cartas.
- **USUARIO:** sección de la web donde el cliente podrá realizar una serie de accines en función de su estado.
  Si no ha iniciado sesión, aparecen las opciones correspondientes para que pueda autenticarse en el sistema, una vez hecho esto, se le devolverá al **nivel 0** con una modificación en el **nivel 1** donde se mostrarán las secciones de **Gestión de Mi Cuenta** y las secciones de **Gestión de mazos creados**, **Gestión de mazos publicados** y **Gestión de mazos guardados**, que redirigirá al **Listado de Mazos** con los diferentes contenidos respectivamente, y a su vez podrá redirigir al **Creador de mazos** para editar los mazos del usuario.



(a) *Header cuando el usuario tiene un login válido*



(b) Header cuando el usuario no tiene un *login* válido

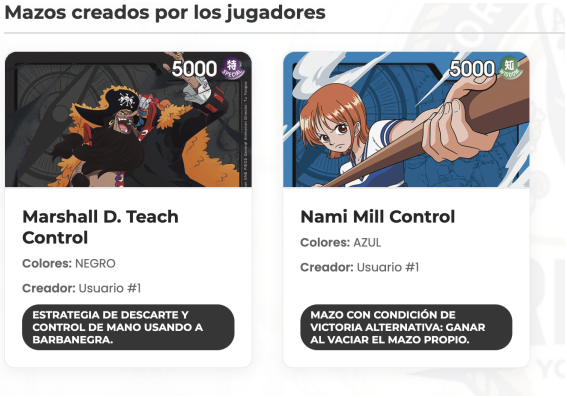
- **TCG:** El apartado TCG mostrará la posible ampliación a futuro de la página web que cambiará el contexto del juego de cartas referido en toda la página.

8.3 Nivel 2: visualizadores y flujos verticales

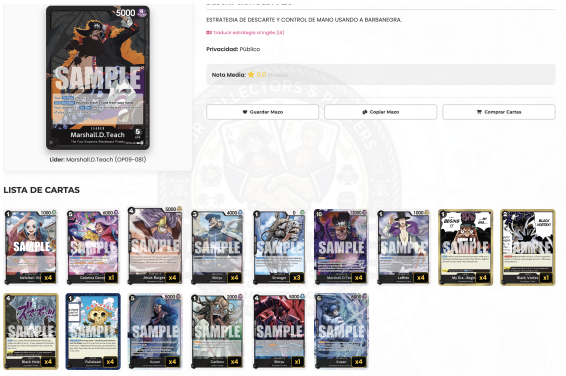
En él encontramos las acciones que requieren 2 click por parte del usuario, donde ya se llega a una funcionalidad en concreto de la página, vamos a describir cada uno de estos apartados:



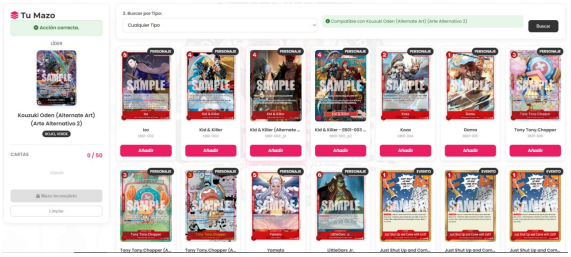
(a) El **visualizador de carta** muestra los detalles concreto de una carta seleccionada por el usuario.



(b) Misma estructura para los *comunity* y *meta decks*, se muestra una lista de los mazos de cada sección con una pequeña descripción y la imagen del líder de cada uno de ellos



(a) El usuario podrá consultar todas las cartas de un mazo, con las recomendaciones de cantidad de copias de cada una de ellas.



(b) Vista para la creación de los mazos con un filtro por líder para que sólo se muestren las cartas compatibles con el arquetipo y color del mismo.



Figura 23: Estructura social, tomando como ejemplo el apartado de Guías

En el apartado de *Social* de la web, tenemos 3 categorías, **guías**, **noticias** y **torneos**, las cuales siguen una estructura similar, una entrada principal con el contenido más representativo del momento, y una lista de entradas las cuales redirigen a sitios externos.
Hemos tomado la decisión de descartar la redirección, puesto que consideramos que supone mucho trabajo para los administradores del sitio el tener que estar constantemente redactando artículos cuando ya hay medios dedicados a cubrir cada uno de estos campos. Sin embargo, lo que sí que harán los responsables, será un filtrado para tener sólo información relevante y actualizada, ya que estar informado de un TCG es parte fundamental del ciclo de vida del jugador.

8.4 Flujos de navegación

Ya tenemos clara la estructura y aspecto visual de la página, así que ahora mostraremos unos casos de ejemplo de las acciones o menús necesarios para que un usuario pueda realizar las acciones principales definidas en el punto 5.1

8.4.1 Consulta de mazos META

El usuario accede página Inicial, desde allí en el *header* se encontrará con varios desplegables, entre ellos *DECKS* que mostrará varias opciones, una de ellas será *META decks*.

Página Inicial → DECKS → *META decks*

8.4.2 Consultar noticias

El usuario, desde el *header*, accede al apartado SOCIAL, en el menú desplegable, seleccionará *NEWS*.

Página Inicial → SOCIAL → Ver Noticias → NEWS

8.4.3 Consulta de cartas específica

Desde el *header*, el usuario selecciona *CARDS* Esto le lleva al catálogo general, donde tras aplicar filtros y buscar la carta deseada, hace clic sobre ella para abrir una vista detallada con toda su información.

Página Inicial → CARDS → Selección de carta → Visualizador de carta

8.4.4 Consultar los detalles de una carta incluida en un mazo

Mientras el usuario estudia la composición de una baraja, se encuentra con una carta que no conoce y necesita ver sus atributos, efectos o precio de mercado.
Partimos desde el visualizador del mazo, el usuario hace clic sobre una carta específica del listado, la web le redirigirá directamente la carta específica mediante el Visualizador de carta, sin tener que buscarla manualmente en el catálogo.

Página Inicial → *DECKS* → Listado mazos X → Visualizador del mazo → Visualizador de carta

8.4.5 Consulta de una de las guías de juego

Desde el *header* el usaario se dirige a la sección *SOCIAL*, dentro del menú despegable, selecciona *GUIDES*, tras lo cual, el sistema mostrará el Listado de guías filtrando la base de datos para enseñar únicamente el contenido de aprendizaje, permitiendo al usuario acceder a las explicaciones.

Página Inicial → SOCIAL → GUIDES → Listado de guías

8.5 Base de datos

Para esta parte del desarrollo empezamos a implementar una bbdd, en concreto nos decantamos por MariaDB a través de XAMPP para facilitar el trabajo de gestión y diseño, ya que permite usar phpMyAdmin desde navegadores web y gestionar la base de datos de manera visual y exportar o importar los scripts de creación e inserción de datos, además de otras herramientas útiles (como el meter archivos .csv).

8.6 Cambios del Modelo de Dominio a Entidad-Relación

Para traducir el modelo de dominio realizado en la práctica 2 al **modelo entidad-relación** se han efectuado los siguientes cambios:

- **Usuario**: Se ha traducido la relación de *Usuario* como clase padre de *Jugador* y *Admin* a una sola tabla `Usuario` y un atributo ROL que especifique qué tipo de usuario es.
- **Mazo**: Al igual que con *Usuario*, la clase *Mazo* se ha traducido a una única tabla `Mazo` y un atributo ES_OFICIAL para hacer la distinción.
- **Relaciones *:***: Las relaciones muchos a muchos entre *Jugador*-**GUARDA**-*Mazo*, *Mazo*-**CONTIENE**-*Carta* y *Jugador*-**VALORA**-*Mazo* se han traducido como las clases intermedias `GUARDA_MAZO`, `CONTIENE_CARTA` y `VALORACION` respectivamente.

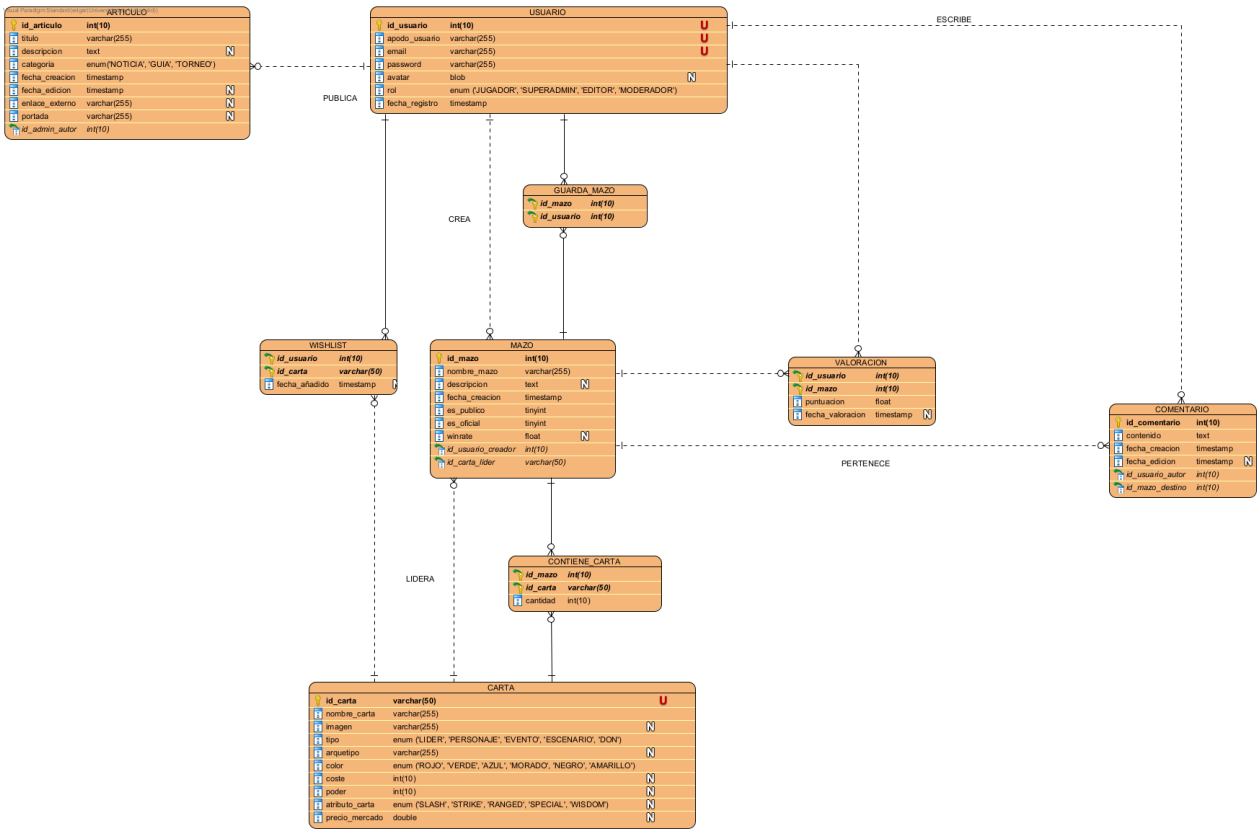


Figura 24: Entidad-Relacion de la base de datos

8.7 Estructura de la Base de Datos

Nuestra base de datos (`cardpartner_db`) está diseñada bajo un modelo relacional orientado a la gestión de una comunidad de juegos de cartas coleccionables (TCG). Su arquitectura se divide en cuatro pilares fundamentales:

8.7.1 Componentes Principales

- **Gestión de Usuarios y Roles:** la entidad `USUARIO` centraliza el acceso y la seguridad. Empleamos un sistema de roles (*Jugador, Editor, Moderador, Superadmin*), el cual es una traducción directa de los niveles de privilegios en el sistema.

- **Catálogo de Cartas y Mazos:**
  - `CARTA`: actúa como el diccionario técnico del juego, almacenando atributos estáticos como coste, poder, color y tipo.
  - `MAZO`: representa la agrupación lógica de cartas creada por un usuario, vinculada obligatoriamente a una carta líder.
- **Módulo de Comunidad:** la interacción social se articula a través de las tablas `VALORACION`, `COMENTARIO` y `GUARDA_MAZO`, permitiendo la retroalimentación y la creación de bibliotecas personales.
- **Sistema de Contenidos:** la entidad `ARTICULO` permite a los administradores gestionar noticias, guías y torneos, manteniendo la plataforma actualizada.

8.8 Resumen de Relaciones y Cardinalidad

Teniendo en claro la estructura de nuestra bbdd, pasaremos a explicar las relaciones entre entidades y las cardinales de las mismas.

Relación	Tipo	Descripción
Usuario → Mazo	1:N	Un usuario es autor de múltiples mazos.
Mazo ↔ Carta	N:M	Relación gestionada por `CONTIENE_CARTA` con cantidades.
Usuario ↔ Mazo	N:M	Biblioteca personal y favoritos (`GUARDA_MAZO`).
Usuario → Articulo	1:N	Relación de autoría para perfiles administrativos.

Cuadro 6: Resumen de la Arquitectura de Datos

8.9 APIs empleadas en nuestro sistema

Uno de los aspectos que teníamos en mente para aplicar respecto a entregas anteriores era el uso de APIs (*Application Programming Interface*), la cual nos otorga la posibilidad de comunicarnos con otra aplicación de una manera sencilla, permitiéndonos introducir funcionalidades extra a *Card Partner* sin tener que desarrollarlas nosotros mismos, consiguiendo así ahorrarnos tiempo y generar menos archivos.

8.9.1 API encargada de la obtención de cartas

Este era uno de los puntos pendientes de la anterior entrega, ya que como se ha podido ver en la figura **??** teníamos una muestra limitada de cartas.
Teniendo más tiempo, hemos implementado un código (se puede ver en la sección A de los anexos) el cuál se encarga de hacer el papel de motor de sincronización de la plataforma, encargándose de generar el catálogo completo de cartas desde una *API REST* externa (empleamos la librería Google **GSON**).

El funcionamiento es simple, el sistema establece una conexión HTTP, descarga el archivo JSON correspondiente y lo procesa de manera iterativa, estableciendo unos *timeouts* y métodos de *parseo*, evitando fallos críticos del sistema al procesar nodos JSON nulos, variables faltantes o formatos numéricos inconsistentes, asegurando que solo se persista información limpia y válida. También traduce datos al castellano y nos permite añadir cartas que tienen más de un color.

Una de las partes más tediosas de la práctica son los alter o artes alternativos, cartas que comparten el mismo id y los mismos atributos, variando únicamente el arte que muestran. Como no se pueden almacenar cartas con un mismo ID en la base de datos Nuestro código altera dinámicamente el identificador (añadiendo sufijos indexados) y el título de la carta, evitando así colisiones de claves primarias.

8.9.2 API de traducción

La clase *GeminiAPI* (la cuál se puede consultar en la sección B de los anexos) actúa como un servicio de traducción inteligente, integrándose mediante peticiones HTTP con la API de Groq y el modelo Llama-3, mandando prompts a la LLM y obteniendo las respuestas de manera casi inmediata. Hacemos especial hincapié en la robustez de las respuestas, para que no aparezcan caracteres raro (como puede ser la traducción de una ñ o una tilde si el texto está en inglés) empleando JSONs para la comunicación con el servicio e implementando un mecanismo para que, si la *IA* no responde, se muestre el texto original en vez de un mensaje de error.

Este módulo adapta dinámicamente sus prompts en función del contexto del texto original, permitiéndonos entrenar este modelo para que utilice jerga específica de TCG (como *Ramp* o *Rested*), o que mantenga un tono natural en los comentarios de los usuarios.

8.10 Descripción de la funcionalidad implementada y justificación de la elección

8.10.1 Gestión de usuarios y perfil

Desarrollamos un sistema de usuarios capaz de distinguir entre visitantes, jugadores registrados y administradores.
Los visitantes pueden consultar contenido público, como la lista de cartas, noticias, guías, torneos y mazos públicos mientras que los jugadores registrados disponen de acceso completo a todo el contenido público con el añadido de un perfil personalizado donde podrán gestionar sus datos, cambiar la contraseña, crear y guardar mazos, comentar y valorar los mazos de la comunidad.
Por último, los administradores tienen privilegios para publicar mazos oficiales y artículos, así como mantener el contenido de la web actualizado.

Hemos decidido centralizar toda la gestión de datos en la sección de **Perfil**, permitiendo a los usuarios acceder a sus activos y configuraciones desde un único punto. Esta elección garantiza que la administración de la información sea intuitiva y accesible, evitando dispersión en múltiples páginas.

8.10.2 Catálogo de cartas y filtros

Para manejar el amplio repertorio de cartas que tiene el TCG de *One Piece* hemos desarrollado filtros avanzados por nombre, color, tipo, coste y rareza, basándonos en la necesidad de los jugadores de localizar rápidamente cartas específicas y planificar la construcción de sus mazos de forma eficiente.
A mayores la carta nos muestra el precio de mercado por el cual se vende en las plataformas online.

8.10.3 Gestión y creación de mazos

El constructor de mazos o *Deckbuilder* ha sido implementado con ventanas emergentes modales que permiten filtrar y asignar la carta líder y así poder configurar la composición completa de un mazo, donde se aplica un filtro automático para mostrar sólo aquellas cartas compatibles por color y/o arquetipo con el líder.

En este campo tenemos dos tipos de mazos, los oficiales o *META* y los comunitarios, diferenciados a través de atributos como `es_publico` o `winrate` para diferencialos. Esto permite que los jugadores puedan crear, guardar y consultar mazos de manera flexible, mientras que los mazos oficiales permanecen verificados y orientados a la escena competitiva.

Además, se han creado vistas de detalle de mazo que muestran el precio total, estadísticas y cartas contenidas, facilitando la toma de decisiones al jugador y mejorando la experiencia de navegación.

8.10.4 Comunidad e interacción

Se ha integrado un sistema de comentarios y valoraciones en cada mazo, permitiendo que los usuarios registrados pueden publicar sus opiniones, sugerir cambios y puntuar mazos creados por otros usuarios.

8.10.5 Social

Como hemos mencionado en puntos anteriores, la sección de noticias, guías y torneos será gestionada por los administradores permitiéndonos mantener una calidad en los contenidos que se muestren en nuestra web consolidando a **Card Partner** como un punto de referencia dentro de la comunidad *TCG*.

8.10.6 Sección TCG

Al inicio de este documento comentábamos que íbamos a emplear el juego de cartas de *One Piece* como referencia, pero consideramos que si la fórmula funcionaba, adentrarnos en el resto de *TCGs* sería una tarea sencilla, ya que simplemente habría que adaptar lo que ya tenemos a cada una de las mecánicas, por eso hemos dejado esta sección donde el usuario podrá encontrarse con los principales juegos del mercado.



Figura 25: *Hub*de selección entre los TCGs

8.11 Justificación de la elección

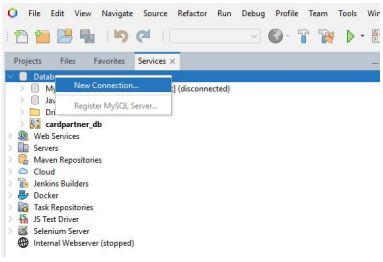
Hemos optado por implementar estas funcionalidades priorizando:

- Centralización de información y herramientas, evitando que los jugadores tengan que usar múltiples webs externas.
- Claridad y facilidad de uso, con un panel de perfil único y vistas modales en el constructor de mazos.
- Escalabilidad, permitiendo agregar nuevas cartas, mazos o secciones sin necesidad de reescribir la estructura base del sistema.
- Interacción social, fomentando la participación de la comunidad mediante comentarios y valoraciones.
- Coherencia con el modelo de dominio y la base de datos, asegurando que la estructura de información sea consistente y fácil de mantener.

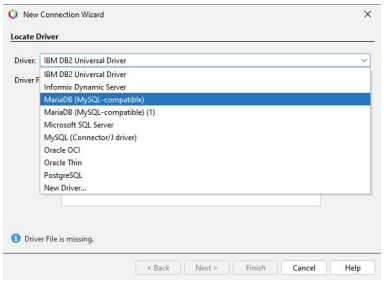
Con estas decisiones, **Card Partner** ofrece una plataforma funcional, intuitiva y completa, que cubre las necesidades más importantes de los jugadores de TCG y justifica las elecciones de diseño realizadas en el desarrollo del sistema.

8.12 Instrucciones de ejecución del código

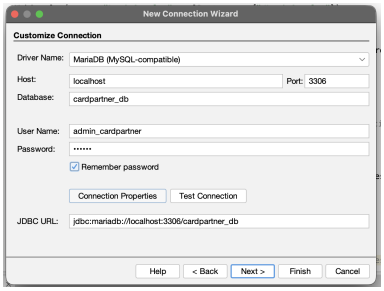
- Ejecutar el script de *setup_db* (que se encuentran en *CardPartner/sql/* o en el directorio raiz de SSW si se mira desde el repositorio de GitHub)
- Realizar la conexión con la bbdd en el Apache NetBeans



(a) Vamos al menú de *services* del editor



(b) En el menú desplegable seleccionamos nuestra base de datos y buscamos el driver que se encuentra en la carpeta lib de nuestro proyecto



(c) Introducimos el nombre de nuestra base de datos (cardpartner_db) con su usuario (admin_cardpartner) y la contraseña (123456)

- Vamos al menú de Files del editor
- Dentro de este menú, buscamos la carpeta que se llame sql
- Ejecutamos los scripts en orden para tener toda la información necesaria en nuestra base de datos.
- Ejecutamos la aplicación.

8.12.1 Usuarios para login

Para la funcionalidad de inicio de sesion, se han creado los siguientes usuarios en la base de datos:

Apodo	Email	Password
admin	admin@test.com	123456
sergio	sergio@test.com	123456
marcos	marcos@test.com	123456
alvaro	admin@cardpartner.com	123456
edgar	edgar@test.com	123456
xabi	xabi@test.com	123456

Cuadro 7: Credenciales de usuarios para pruebas de sistema

9. Declaración de Uso de Inteligencia Artificial

Para esta segunda entrega, se ha usado la IA Gemini como herramienta para dudas y mejoras del archivo LaTeX, revisión de los contenidos y evaluación de los mismos y diseño iterativo para mejorar lo ya realizado. Además, se ha usado Gemini para la generación del logo del proyecto.
Ejemplos de *prompts* realizados a lo largo de toda la entrega:

- 'Cómo podemos realizar un menú desplegable en html a partir de un botón'
- 'Cómo podemos hcer un enlace en una imagen'
- 'Cómo podemos hacer un popup para cambiar la contraseña'
- 'Cómo funciona un hover en html'
- 'Redimensionar los botones para que ocupen una parte proporcional del encabezado'
- 'En css por qué me sale un link subrayado'
- 'Como puedo hacer un popup con sobreado'
- 'La estructura de un proyecto html cuál es'
- 'Entre la idea X y la idea Y, dime posibles ventajas y desventajas de cada uno y cuál se adaptaría mejor al proyecto y por qué'
- 'Cómo funciona un servlet en java'
- 'Que son los DAOs, DTOs y Servlets en java y como es el flujo de trabajo entre ellos'
- 'Como hacer una conexión de MariaDB en ApacheNetBeans'
- 'Haciendo push, a mis compañeros no les aparece la bbdd ¿es normal?'
- 'Cómo puedo automatizar la instalación y creación de una bbdd con MariaDB'
- 'Me puedes revisar los *scripts* de creación de la base de datos'
- 'Traduceme este *script* de creación de bbdd para Windows en un .bat'
- 'Cómo puedo testear que la base de datos se ha rellenado de manera correcta con los *scrpts* sql'
- 'Cómo me puedo saltar la política CORS para hacer peticiones a la web de bandai nanco sobre cartas del tcg de One Piece'
- 'Cómo puedo cambiar la numeración de una página para sean botones y mejorar la navegabilidad por una lista de cartas'
- 'Ayudame a mejorar el botón de filtros de la vista cartas'
- '¿Cómo puedo añadir un mensaje de aviso que indique que no se puede acceder a esta vista sin estar loggeado?'
- 'No logro que se apliquen los filtros en las cartas, ¿puedes echarme una mano?'

Anexo

Adjuntamos el código de las modificamos más importantes para que se tenga un acceso más rápido

A. API de las cartas

Listing 1: Clase completa OptcgAPI.

```
package controladores;

import datos.CartaDAO;
import modelo.Carta;
import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.HashMap;
import java.util.ArrayList;
import java.util.List;

public class OptcgAPI {

    private static final String API_URL = "https://www.optcgapi.com/api/allSetCards/";

    public static String sincronizarBaseDeDatos() {
        try {
            System.out.println("Iniciando conexión con OPTCG API (Versión Definitiva con Artes Alternativos)...");
            URL url = new URL(API_URL);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            conn.setRequestProperty("User-Agent", "Mozilla/5.0");

            // 15 segundos para que la API acepte la conexión
            conn.setConnectTimeout(15000);
            // 60 segundos de tiempo máximo de "silencio" (espera entre paquetes de datos)
            conn.setReadTimeout(40000);

            if (conn.getResponseCode() != 200) {
                return "Error de conexión: HTTP " + conn.getResponseCode();
            }

            // 1. Leer el JSON gigante
            BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream(), "UTF-8"));
            StringBuilder responseStr = new StringBuilder();
            String line;
            while ((line = br.readLine()) != null) {
                responseStr.append(line);
            }
            br.close();

            // 2. Parsear el texto con GSON
            JsonArray jsonArray = JsonParser.parseString(responseStr.toString()).getAsJsonArray();
            CartaDAO cartaDAO = new CartaDAO();
            int cartasInsertadas = 0;

            // Un "cerebro" temporal para recordar qué cartas hemos visto ya hoy (Artes Alternativos)
```

```
54              HashMap<String, Integer> conteoCartas = new HashMap<>();
55
56              // 3. Recorrer el Array
57              for (JsonElement element : jsonArray) {
58                  JsonObject jsonCarta = element.getAsJsonObject();
59
60                  // Evitar cartas sin ID o valores nulos de la API
61                  if (!jsonCarta.has("card_set_id") || jsonCarta.get("card_set_id").isJsonNull
    ()) {
62                      continue;
63                  }
64                  String idBase = jsonCarta.get("card_set_id").getAsString();
65                  if (idBase.equals("NULL") || idBase.isEmpty()) {
66                      continue;
67                  }
68
69                  // Lógica de Artes Alternativos
70                  int apariciones = conteoCartas.getOrDefault(idBase, 0);
71                  String idFinal = idBase;
72                  String nombreOriginal = obtenerTexto(jsonCarta, "card_name", "Desconocido");
73                  String nombreFinal = nombreOriginal;
74
75                  if (apariciones > 0) {
76                      // Si ya existe en esta pasada, le ponemos el sufijo "_p1", "_p2", etc.
77                      idFinal = idBase + "_p" + apariciones;
78                      nombreFinal = nombreOriginal + "␣(Arte␣Alternativo␣" + apariciones + ")"
    ;
79                  }
80
81                  // Sumamos 1 al contador para esta carta específica
82                  conteoCartas.put(idBase, apariciones + 1);
83
84                  Carta carta = new Carta();
85                  carta.setIdCarta(idFinal);
86                  carta.setNombreCarta(nombreFinal);
87
88                  carta.setImagen(obtenerTexto(jsonCarta, "card_image", null));
89                  carta.setArquetipo(obtenerTexto(jsonCarta, "sub_types", null));
90
91                  // Traducción y mapeo (NUEVO MAPEO DE COLOR)
92                  carta.setTipo(mapearTipo(obtenerTexto(jsonCarta, "card_type", "")));
93                  carta.setColor(extraerYTraducirColores(jsonCarta.get("card_color")));
94                  carta.setAtributoCarta(mapearAtributo(obtenerTexto(jsonCarta, "attribute", "
    ")));
95
96                  // Conversión segura de números
97                  carta.setCoste(obtenerEnteroSeguro(obtenerTexto(jsonCarta, "card_cost", "0")
    ));
98                  carta.setPoder(obtenerEnteroSeguro(obtenerTexto(jsonCarta, "card_power", "0"
    )));
99                  carta.setPrecio((float) obtenerNumero(jsonCarta, "market_price", 0.0));
100
101                 // 4. Guardar en Base de Datos
102                 cartaDAO.insertarOActualizarCarta(carta);
103                 cartasInsertadas++;
104             }
105
106             return "¡Éxito!␣Se␣han␣sincronizado␣" + cartasInsertadas + "␣cartas␣desde␣la␣API
    ␣incluyendo␣Artes␣Alternativos.";
107
108         } catch (Exception e) {
109             e.printStackTrace();
110             return "Error␣crítico␣durante␣la␣sincronización:␣" + e.getMessage();
111         }
112     }
113
114     // ========================================
115     // ESCUDOS ANTIMISSILES (Métodos de Seguridad)
```

```
116     // ========================================
117
118     private static String obtenerTexto(JsonObject json, String clave, String valorPorDefecto
        ) {
119         if (json.has(clave) && !json.get(clave).isJsonNull()) {
120             return json.get(clave).getAsString();
121         }
122         return valorPorDefecto;
123     }
124
125     private static double obtenerNumero(JsonObject json, String clave, double
        valorPorDefecto) {
126         if (json.has(clave) && !json.get(clave).isJsonNull()) {
127             try { return json.get(clave).getAsDouble(); }
128             catch (Exception e) { return valorPorDefecto; }
129         }
130         return valorPorDefecto;
131     }
132
133     private static int obtenerEnteroSeguro(String valor) {
134         if (valor == null || valor.equals("NULL") || valor.isEmpty()) return 0;
135         try {
136             return Integer.parseInt(valor.replaceAll("[^0-9-]", ""));
137         } catch (NumberFormatException e) {
138             return 0;
139         }
140     }
141
142     // ========================================
143     // MÉTODOS DE TRADUCCIÓN A NUESTRA BD
144     // ========================================
145
146     private static String mapearTipo(String tipoEn) {
147         if (tipoEn == null || tipoEn.isEmpty()) return "PERSONAJE";
148         return switch (tipoEn.toUpperCase()) {
149             case "LEADER" -> "LIDER";
150             case "CHARACTER" -> "PERSONAJE";
151             case "EVENT" -> "EVENTO";
152             case "STAGE" -> "ESCENARIO";
153             default -> "PERSONAJE";
154         };
155     }
156
157     // NUEVO: Escáner inteligente de colores que soporta múltiples colores sin colapsar
158     private static String extraerYTraducirColores(JsonElement colElement) {
159         if (colElement == null || colElement.isJsonNull()) return "MULTICOLOR";
160
161         List<String> coloresEncontrados = new ArrayList<>();
162
163         // Si la API devuelve un array: ["Red", "Green"]
164         if (colElement.isJsonArray()) {
165             JsonArray arr = colElement.getAsJsonArray();
166             for (JsonElement c : arr) {
167                 if (c != null && !c.isJsonNull()) {
168                     aniadirColor(c.getAsString(), coloresEncontrados);
169                 }
170             }
171         } else {
172             // Si devuelve un solo String: "Red/Green" o "Red Green"
173             String colorStr = colElement.getAsString();
174             aniadirColor(colorStr, coloresEncontrados);
175         }
176
177         if (coloresEncontrados.isEmpty()) return "MULTICOLOR";
178
179         // Une los colores encontrados con comas. Ej: "ROJO, VERDE"
180         return String.join(", ", coloresEncontrados);
181     }
```

```
182
183     // Función auxiliar que busca palabras clave en inglés y añade la española a la lista
184     private static void aniadirColor(String texto, List<String> lista) {
185         if (texto == null || texto.isEmpty()) return;
186         String upper = texto.toUpperCase();
187
188         if (upper.contains("RED") && !lista.contains("ROJO")) lista.add("ROJO");
189         if (upper.contains("GREEN") && !lista.contains("VERDE")) lista.add("VERDE");
190         if (upper.contains("BLUE") && !lista.contains("AZUL")) lista.add("AZUL");
191         if (upper.contains("YELLOW") && !lista.contains("AMARILLO")) lista.add("AMARILLO");
192         if (upper.contains("PURPLE") && !lista.contains("MORADO")) lista.add("MORADO");
193         if (upper.contains("BLACK") && !lista.contains("NEGRO")) lista.add("NEGRO");
194     }
195
196     private static String mapearAtributo(String attrEn) {
197         if (attrEn == null || attrEn.equals("NULL") || attrEn.isEmpty()) return null;
198
199         String upper = attrEn.toUpperCase().trim();
200         if (upper.equals("SLASH") || upper.equals("STRIKE") || upper.equals("RANGED") ||
201             upper.equals("SPECIAL") || upper.equals("WISDOM")) {
202             return upper;
203         }
204
205         return null;
206     }
207 }
```

B. API de traducción

Listing 2: Clase completa OptcgAPI.

```
1  package controladores;
2
3  import datos.CartaDAO;
4  import modelo.Carta;
5  import com.google.gson.JsonArray;
6  import com.google.gson.JsonElement;
7  import com.google.gson.JsonObject;
8  import com.google.gson.JsonParser;
9
10 import java.io.BufferedReader;
11 import java.io.InputStreamReader;
12 import java.net.HttpURLConnection;
13 import java.net.URL;
14 import java.util.HashMap;
15 import java.util.ArrayList;
16 import java.util.List;
17
18 public class OptcgAPI {
19
20     private static final String API_URL = "https://www.optcgapi.com/api/allSetCards/";
21
22     public static String sincronizarBaseDeDatos() {
23         try {
24             System.out.println("Iniciando conexión con OPTCG API (Versión Definitiva con
    Artes Alternativos)...");
25             URL url = new URL(API_URL);
26             HttpURLConnection conn = (HttpURLConnection) url.openConnection();
27             conn.setRequestMethod("GET");
28             conn.setRequestProperty("User-Agent", "Mozilla/5.0");
29
30             // 15 segundos para que la API acepte la conexión
31             conn.setConnectTimeout(15000);
32             // 60 segundos de tiempo máximo de "silencio" (espera entre paquetes de datos)
33             conn.setReadTimeout(40000);
```

```
34
35          if (conn.getResponseCode() != 200) {
36              return "Error de conexión: HTTP " + conn.getResponseCode();
37          }
38
39          // 1. Leer el JSON gigante
40          BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream
    (), "UTF-8"));
41          StringBuilder responseStr = new StringBuilder();
42          String line;
43          while ((line = br.readLine()) != null) {
44              responseStr.append(line);
45          }
46          br.close();
47
48          // 2. Parsear el texto con GSON
49          JsonArray jsonArray = JsonParser.parseString(responseStr.toString()).
    getAsJsonArray();
50          CartaDAO cartaDAO = new CartaDAO();
51          int cartasInsertadas = 0;
52
53          // Un "cerebro" temporal para recordar qué cartas hemos visto ya hoy (Artes
    Alternativos)
54          HashMap<String, Integer> conteoCartas = new HashMap<>();
55
56          // 3. Recorrer el Array
57          for (JsonElement element : jsonArray) {
58              JsonObject jsonCarta = element.getAsJsonObject();
59
60              // Evitar cartas sin ID o valores nulos de la API
61              if (!jsonCarta.has("card_set_id") || jsonCarta.get("card_set_id").isJsonNull
    ()) {
62                  continue;
63              }
64              String idBase = jsonCarta.get("card_set_id").getAsString();
65              if (idBase.equals("NULL") || idBase.isEmpty()) {
66                  continue;
67              }
68
69              // Lógica de Artes Alternativos
70              int apariciones = conteoCartas.getOrDefault(idBase, 0);
71              String idFinal = idBase;
72              String nombreOriginal = obtenerTexto(jsonCarta, "card_name", "Desconocido");
73              String nombreFinal = nombreOriginal;
74
75              if (apariciones > 0) {
76                  // Si ya existe en esta pasada, le ponemos el sufijo "_p1", "_p2", etc.
77                  idFinal = idBase + "_p" + apariciones;
78                  nombreFinal = nombreOriginal + " (Arte Alternativo " + apariciones + ")"
    ;
79              }
80
81              // Sumamos 1 al contador para esta carta específica
82              conteoCartas.put(idBase, apariciones + 1);
83
84              Carta carta = new Carta();
85              carta.setIdCarta(idFinal);
86              carta.setNombreCarta(nombreFinal);
87
88              carta.setImagen(obtenerTexto(jsonCarta, "card_image", null));
89              carta.setArquetipo(obtenerTexto(jsonCarta, "sub_types", null));
90
91              // Traducción y mapeo (NUEVO MAPEO DE COLOR)
92              carta.setTipo(mapearTipo(obtenerTexto(jsonCarta, "card_type", "")));
93              carta.setColor(extraerYTraducirColores(jsonCarta.get("card_color")));
94              carta.setAtributoCarta(mapearAtributo(obtenerTexto(jsonCarta, "attribute", "
    ")));
95
```

```
96                  // Conversión segura de números
97                  carta.setCoste(obtenerEnteroSeguro(obtenerTexto(jsonCarta, "card_cost", "0")
    ));
98                  carta.setPoder(obtenerEnteroSeguro(obtenerTexto(jsonCarta, "card_power", "0"
    )));
99                  carta.setPrecio((float) obtenerNumero(jsonCarta, "market_price", 0.0));
100
101                 // 4. Guardar en Base de Datos
102                 cartaDAO.insertarOActualizarCarta(carta);
103                 cartasInsertadas++;
104             }
105
106             return "¡Éxito!␣Se␣han␣sincronizado␣" + cartasInsertadas + "␣cartas␣desde␣la␣API
    ␣incluyendo␣Artes␣Alternativos.";
107
108         } catch (Exception e) {
109             e.printStackTrace();
110             return "Error␣crítico␣durante␣la␣sincronización:␣" + e.getMessage();
111         }
112     }
113
114     // ==========================================
115     // ESCUDOS ANTIMISSILES (Métodos de Seguridad)
116     // ==========================================
117
118     private static String obtenerTexto(JsonObject json, String clave, String valorPorDefecto
    ) {
119         if (json.has(clave) && !json.get(clave).isJsonNull()) {
120             return json.get(clave).getAsString();
121         }
122         return valorPorDefecto;
123     }
124
125     private static double obtenerNumero(JsonObject json, String clave, double
    valorPorDefecto) {
126         if (json.has(clave) && !json.get(clave).isJsonNull()) {
127             try { return json.get(clave).getAsDouble(); }
128             catch (Exception e) { return valorPorDefecto; }
129         }
130         return valorPorDefecto;
131     }
132
133     private static int obtenerEnteroSeguro(String valor) {
134         if (valor == null || valor.equals("NULL") || valor.isEmpty()) return 0;
135         try {
136             return Integer.parseInt(valor.replaceAll("[^0-9-]", ""));
137         } catch (NumberFormatException e) {
138             return 0;
139         }
140     }
141
142     // ==========================================
143     // MÉTODOS DE TRADUCCIÓN A NUESTRA BD
144     // ==========================================
145
146     private static String mapearTipo(String tipoEn) {
147         if (tipoEn == null || tipoEn.isEmpty()) return "PERSONAJE";
148         return switch (tipoEn.toUpperCase()) {
149             case "LEADER" -> "LIDER";
150             case "CHARACTER" -> "PERSONAJE";
151             case "EVENT" -> "EVENTO";
152             case "STAGE" -> "ESCENARIO";
153             default -> "PERSONAJE";
154         };
155     }
156
157     // NUEVO: Escáner inteligente de colores que soporta múltiples colores sin colapsar
158     private static String extraerYTraducirColores(JsonElement colElement) {
```

```
159         if (colElement == null || colElement.isJsonNull()) return "MULTICOLOR";
160
161         List<String> coloresEncontrados = new ArrayList<>();
162
163         // Si la API devuelve un array: ["Red", "Green"]
164         if (colElement.isJsonArray()) {
165             JsonArray arr = colElement.getAsJsonArray();
166             for (JsonElement c : arr) {
167                 if (c != null && !c.isJsonNull()) {
168                     aniadirColor(c.getAsString(), coloresEncontrados);
169                 }
170             }
171         } else {
172             // Si devuelve un solo String: "Red/Green" o "Red Green"
173             String colorStr = colElement.getAsString();
174             aniadirColor(colorStr, coloresEncontrados);
175         }
176
177         if (coloresEncontrados.isEmpty()) return "MULTICOLOR";
178
179         // Une los colores encontrados con comas. Ej: "ROJO, VERDE"
180         return String.join(", ", coloresEncontrados);
181     }
182
183     // Función auxiliar que busca palabras clave en inglés y añade la española a la lista
184     private static void aniadirColor(String texto, List<String> lista) {
185         if (texto == null || texto.isEmpty()) return;
186         String upper = texto.toUpperCase();
187
188         if (upper.contains("RED") && !lista.contains("ROJO")) lista.add("ROJO");
189         if (upper.contains("GREEN") && !lista.contains("VERDE")) lista.add("VERDE");
190         if (upper.contains("BLUE") && !lista.contains("AZUL")) lista.add("AZUL");
191         if (upper.contains("YELLOW") && !lista.contains("AMARILLO")) lista.add("AMARILLO");
192         if (upper.contains("PURPLE") && !lista.contains("MORADO")) lista.add("MORADO");
193         if (upper.contains("BLACK") && !lista.contains("NEGRO")) lista.add("NEGRO");
194     }
195
196     private static String mapearAtributo(String attrEn) {
197         if (attrEn == null || attrEn.equals("NULL") || attrEn.isEmpty()) return null;
198
199         String upper = attrEn.toUpperCase().trim();
200         if (upper.equals("SLASH") || upper.equals("STRIKE") || upper.equals("RANGED") ||
201             upper.equals("SPECIAL") || upper.equals("WISDOM")) {
202             return upper;
203         }
204
205         return null;
206     }
207 }
```